

Lab 1

Perform basic SQL commands like Create, Insert, Select

Database Name: Course_Rollno (Example: MCA_101)

Note: Create all the tables under above database.

Create following tables and insert the data into tables using query as shown below.

DEPOSIT	
Column_Name	DataType
ACTNO	INT
CNAME	VARCHAR(50)
BNAME	VARCHAR(50)
AMOUNT	DECIMAL(8,2)
ADATE	DATETIME

ACTNO	CNAME	BNAME	AMOUNT	ADATE
101	ANIL	VRCE	1000.00	1-3-95
102	SUNIL	AJNI	5000.00	4-1-96
103	MEHUL	KAROLBAGH	3500.00	17-11-95
104	MADHURI	CHANDI	1200.00	17-12-95
105	PRAMOD	M.G. ROAD	3000.00	27-3-96
106	SANDIP	ANDHERI	2000.00	31-3-96
107	SHIVANI	VIRAR	1000.00	5-9-95
108	KRANTI	NEHRU PLACE	5000.00	2-7-95
109	MINU	POWAI	7000.00	10-8-95

BRANCH	
Column_Name	DataType
BNAME	VARCHAR(50)
CITY	VARCHAR(50)

BNAME	CITY
VRCE	NAGPUR
AJNI	NAGPUR
KAROLBAGH	DELHI
CHANDI	DELHI
DHARAMPETH	NAGPUR
M.G. ROAD	BANGLORE
ANDHERI	BOMBAY
VIRAR	BOMBAY
NEHRU PLACE	DELHI
POWAI	BOMBAY

CUSTOMERS	
Column_Name	DataType
CNAME	VARCHAR(50)
CITY	VARCHAR(50)

CNAME	CITY
ANIL	CALCUTTA
SUNIL	DELHI
MEHUL	BARODA
MANDAR	PATNA
MADHURI	NAGPUR
PRAMOD	NAGPUR
SANDIP	SURAT
SHIVANI	BOMBAY
KRANTI	BOMBAY
NAREN	BOMBAY

BORROW	
Column_Name	DataType
LOANNO	INT
CNAME	VARCHAR(50)
BNAME	VARCHAR(50)
AMOUNT	DECIMAL(8,2)

LOANNO	CNAME	BNAME	AMOUNT
201	ANIL	VRCE	1000.00
206	MEHUL	AJNI	5000.00
311	SUNIL	DHARAMPETH	3000.00
321	MADHURI	ANDHERI	2000.00
375	PRAMOD	VIRAR	8000.00
481	KRANTI	NEHRU PLACE	3000.00

SELECT Operation

Part – A:

From the above given tables perform the following queries:

1. Retrieve all data from table DEPOSIT.

SELECT * FROM DEPOSIT

2. Retrieve all data from table BORROW.

SELECT * FROM BORROW

3. Display Account No, Customer Name & Amount from DEPOSIT.

SELECT ACTNO,CNAME,AMOUNT FROM DEPOSIT

4. Display Loan No, Amount from BORROW.

SELECT LOANNO,AMOUNT FROM BORROW

5. Display loan details of all customers who belongs to 'ANDHERI' branch from borrow table.

SELECT LOANNO,AMOUNT FROM BORROW WHERE BNAME='ANDHERI'

6. Give account no and amount of depositor, whose account no is equals to 106 from deposit table.

SELECT ACTNO,AMOUNT FROM DEPOSIT WHERE ACTNO=106

7. Give name of borrowers having amount greater than 5000 from borrow table.

SELECT BNAME FROM BORROW WHERE AMOUNT>5000

8. Give name of customers who opened account after date '1-12-95' from deposit table.

SELECT CNAME FROM DEPOSIT WHERE ADATE>'1-12-95'

9. Display name of customers whose account no is less than 105 from deposit table.

SELECT CNAME FROM DEPOSIT WHERE ACTNO<105

10. Display name of customer who belongs to either 'NAGPUR' or 'DELHI' from customer table. (USE OR & IN)

SELECT CNAME FROM CUSTOMERS WHERE CITY='NAGPUR'OR CITY='DELHI'

SELECT CNAME FROM CUSTOMERS WHERE CITY IN('NAGPUR','DELHI')

11. Display name of customers with branch whose amount is greater than 4000 and account no is less than 105 from deposit table.

SELECT CNAME,BNAME FROM DEPOSIT WHERE AMOUNT>4000 AND ACTNO<105

12. Find all borrowers whose amount is greater than equals to 3000 & less than equals to 8000 from borrow table. (USE AND & BETWEEN)

SELECT CNAME FROM BORROW WHERE AMOUNT>=3000 AND AMOUNT<=8000

SELECT CNAME FROM BORROW WHERE AMOUNT BETWEEN 3000 AND 8000

13. Find all depositors who do not belongs to 'ANDHERI' branch from deposit table.

SELECT CNAME FROM DEPOSIT WHERE BNAME<> 'ANDHERI'

14. Display Account No, Customer Name & Amount of such customers who belongs to 'AJNI', 'KAROLBAGH' Or 'M.G. ROAD' and Account No is less than 104 from deposit table.

SELECT ACTNO,CNAME,AMOUNT FROM DEPOSIT WHERE BNAME IN('AJNI','KAROLBAGH','M.G.ROAD') AND ACTNO<104

15. Display all loan no, customer from borrow table does not belong to 'VIRAR' or 'AJNI' branch. (use NOT IN)

```
SELECT LOANNO,CNAME FROM BORROW WHERE BNAME NOT IN('VIRAR','AJNI')
```

16. Display all the customer's name other than 'MINU' from deposit table (Use: NOT, <>, !=)

```
SELECT CNAME FROM DEPOSIT WHERE NOT CNAME='MINU'
```

```
SELECT CNAME FROM DEPOSIT WHERE CNAME<>'MINU'
```

```
SELECT CNAME FROM DEPOSIT WHERE CNAME!='MINU'
```

17. Display customer name from deposit table whose branch name is not available. (NULL)

```
SELECT CNAME FROM DEPOSIT WHERE BNAME IS NULL
```

18. Retrieve all unique branches using DISTINCT. (Use Branch Table)

```
SELECT DISTINCT BNAME FROM BRANCH
```

19. Retrieve first 50% record from borrow table.

```
SELECT TOP 50 PERCENT * FROM BORROW
```

20. Retrieve first five account number from deposit table.

```
SELECT TOP 5 ACTNO FROM DEPOSIT
```

Part – B:

1. Display all the details of first five customers from deposit table.

```
SELECT TOP 5 * FROM DEPOSIT
```

2. Display all the details of first three depositors from deposit table whose amount is greater than 1000.

```
SELECT TOP 3 * FROM DEPOSIT WHERE AMOUNT>1000
```

3. Display Loan No, Customer Name of first five borrowers whose branch name does not belongs to 'ANDHERI' from borrow table.

```
SELECT TOP 5 LOANNO,CNAME FROM BORROW WHERE BNAME<> 'ANDHERI'
```

4. Select all details with account numbers not in the range 105 to 109 in deposit table.

```
SELECT * FROM DEPOSIT WHERE ACTNO NOT BETWEEN 105 AND 109
```

5. Select all records from BORROW where the amount is greater than 1000 and less than or equal to 7000, and the loan number is between 250 and 600.

```
SELECT * FROM BORROW WHERE AMOUNT>1000 AND AMOUNT<=7000 AND LOANNO BETWEEN 250 AND 600
```

Part – C:

1. Display all the detail of customer who deposited more than 5000 without using * from deposit table.

```
SELECT ACTNO,CNAME,BNAME,AMOUNT,ADATE FROM DEPOSIT WHERE AMOUNT>5000
```

2. Retrieve all unique customer names with city. (Use Customer table)

```
SELECT DISTINCT CNAME,CITY FROM CUSTOMERS
```

	3. Retrieve records from the BORROW table where the loan amount is greater than 3000 and the loan number is not a multiple of 3. SELECT * FROM BORROW WHERE AMOUNT>3000 AND LOANNO % 3 !=0 4. Retrieve records from the DEPOSIT table where amount is greater than 2000 also account number is between 100 and 110 and date is after '1-MAR-1995' or before '27-MAR-1996'. SELECT * FROM DEPOSIT WHERE (AMOUNT > 2000) AND (ACTNO BETWEEN 100 AND 110) AND (ADATE>'1995-03-01' OR ADATE<'1996-03-27') 5. Retrieve all odd/even value loan number from Borrow table SELECT LOANNO FROM BORROW WHERE LOANNO % 2=0 SELECT LOANNO FROM BORROW WHERE LOANNO %2 !=0
Lab 2	Perform SQL queries for Update, Alter, Rename, Delete, Truncate, Drop Update Operation Part – A: From the above given tables perform the following queries (UPDATE Operation): - Update deposit amount of all customers from 3000 to 5000. (Use Deposit Table) UPDATE DEPOSIT SET AMOUNT=50000 WHERE AMOUNT=3000 - Change branch name of ANIL from VRCE to C.G. ROAD. (Use Borrow Table) UPDATE BORROW SET BNAME='C.G.ROAD' WHERE CNAME='ANIL' - Update Account No of SANDIP to 111 & Amount to 5000. (Use Deposit Table) UPDATE DEPOSIT SET ACTNO=111,AMOUNT=5000 WHERE CNAME='SANDIP' - Update amount of KRANTI to 7000. (Use Deposit Table) UPDATE DEPOSIT SET AMOUNT=7000 WHERE CNAME='KRANTI' - Update branch name from ANDHERI to ANDHERI WEST. (Use Branch Table) UPDATE BRANCH SET BNAME='ANDHERI WEST'WHERE BNAME='ANDHERI' - Update branch name of MEHUL to NEHRU PALACE. (Use Deposit Table) UPDATE DEPOSIT SET BNAME='NEHRU PLACE' WHERE CNAME='MEHUL' - Update deposit amount of all depositors to 5000 whose account no between 103 & 107. (Use Deposit Table) UPDATE DEPOSIT SET AMOUNT=5000 WHERE ACTNO BETWEEN 103 AND 107 - Update ADATE of ANIL to 1-4-95. (Use Deposit Table) UPDATE DEPOSIT SET ADATE='1995-4-1' WHERE CNAME='ANIL' - Update the amount of MINU to 10000. (Use Deposit Table) UPDATE DEPOSIT SET AMOUNT=10000 WHERE CNAME='MINU' - Update deposit amount of PRAMOD to 5000 and ADATE to 1-4-96 (Use Deposit Table) UPDATE DEPOSIT SET AMOUNT=50000,ADATE='1996-4-1' WHERE CNAME='PRAMOD'

Part – B:

1. Give 10% Increment in Loan Amount. (Use **Borrow Table**)
UPDATE BORROW
SET AMOUNT = Amount + (Amount*0.1)
2. Customer deposits additional 20% amount to their account, update the same. (Use **Deposit Table**)
UPDATE BORROW
SET AMOUNT = Amount + (Amount*0.2)
3. Increase Amount by 1000 in all the account. (Use **Deposit Table**)
UPDATE DEPOSIT SET AMOUNT = AMOUNT+1000
4. Update the BORROW table to set the amount to 7000 and the branch name to 'CENTRAL' where the customer name is 'MEHUL' and the loan number is even.
UPDATE BORROW SET AMOUNT = 7000 ,BNAME = 'CENTRAL' WHERE CNAME='MEHUL' AND LOANNO%2=0
5. Update the DEPOSIT table to set the date to '2022-05-15' and the amount to 2500 for all accounts in 'VRCE' and with an account number less than 105.
UPDATE DEPOSIT SET ADATE='2022-05-15', AMOUNT =2500 WHERE BNAME='VRCE' AND ACTNO<105

Part – C:

1. Update amount of loan no 321 to *NULL*. (Use **Borrow Table**)
UPDATE BORROW SET AMOUNT=NULL WHERE LOANNO=321
2. Update branch name of KRANTI to *NULL* (Use **Borrow Table**)
UPDATE BORROW SET BNAME=NULL WHERE BNAME='KRANTI'
3. Display the name of borrowers whose Loan number is *NULL*. (Use **Borrow Table**)
SELECT CNAME FROM BORROW WHERE LOANNO IS NULL
4. Display the Borrowers whose having branch. (Use **Borrow Table**)
SELECT CNAME FROM BORROW WHERE BNAME IS NOT NULL
5. Update the Loan Amount to 5000, Branch to VRCE & Customer Name to Darshan whose loan no is 481. (Use **Borrow Table**)
UPDATE BORROW SET AMOUNT=5000,BNAME='VRCE',CNAME='DARSHAN' WHERE LOANNO=481
6. Update the Deposit table and set the date to 01-01-2021 for all the depositor whose amount is less than 2000.
UPDATE DEPOSIT SET ADATE='2021-01-01' WHERE AMOUNT<2000
7. Update the Deposit table and set the date to *NULL* & Branch name to 'ANDHERI' whose Account No is 110.
UPDATE DEPOSIT SET ADATE=NULL ,BNAME='ANDHERI'WHERE ACTNO=110

Alter, Rename Operation

Part – A:

Use Deposit table of lab-1.

DEPOSIT	
Column_Name	DataType
ACTNO	INT
CNAME	VARCHAR(50)
BNAME	VARCHAR(50)
AMOUNT	DECIMAL(8,2)
ADATE	DATETIME

From the above given tables perform the following queries (ALTER, RENAME Operation):

1. Add two more columns City VARCHAR (20) and Pincode INT.
ALTER TABLE DEPOSIT
ADD CITY VARCHAR(20),PINCODE INT
2. Add column state VARCHAR(20).
ALTER TABLE DEPOSIT
ADD STATE VARCHAR(20)
3. Change the size of CNAME column from VARCHAR (50) to VARCHAR (35).
ALTER TABLE DEPOSIT
ALTER COLUMN CNAME VARCHAR(35)
4. Change the data type DECIMAL to INT in amount Column.
ALTER TABLE DEPOSIT
ALTER COLUMN AMOUNT INT
5. Delete Column City from the DEPOSIT table.
ALTER TABLE DEPOSIT
DROP COLUMN CITY
6. Rename Column ActNo to ANO.
SP_RENAME 'DEPOSIT.ACTNO','ANO'
7. Change name of table DEPOSIT to DEPOSIT_DETAIL.
SP_RENAME 'DEPOSIT','DEPOSIT_DETAIL'

Part – B:

1. Rename Column ADATE to AOPENDATE OF DEPOSIT_DETAIL table.
SP_RENAME 'DEPOSIT_DETAIL.ADATE','AOPENDATE'
2. Delete Column AOPENDATE from the DEPOSIT_DETAIL table.
ALTER TABLE DEPOSIT_DETAIL
DROP COLUMN AOPENDATE

- Rename Column CNAME to CustomerName.
SP_RENAME 'DEPOSIT_DETAIL.CNAME','CUSTOMERNAME'

- Add Column country.
ALTER TABLE DEPOSIT_DETAIL
ADD COUNTRY VARCHAR(50)

Part – C:

Create following table using query according to the definition.

STUDENT_DETAIL	
Column_Name	Data Type
Enrollment_No	VARCHAR(20)
Name	VARCHAR(25)
CPI	DECIMAL(5,2)
Birthdate	DATETIME

From the above given tables perform the following queries (ALTER, RENAME Operation):

- Add two more columns City VARCHAR (20) (Not null) and Backlog INT (Null).
ALTER TABLE STUDENT_DETAIL
ADD CITY VARCHAR(20) NOT NULL, BACKLOG INT NULL
- Add column department VARCHAR (20) Not Null.
ALTER TABLE STUDENT_DETAIL
ADD DEPARTMENT VARCHAR(20) NOT NULL
- Change the size of NAME column of student_detail from VARCHAR (25) to VARCHAR (35).
ALTER TABLE STUDENT_DETAIL
ALTER COLUMN NAME VARCHAR(35)
- Change the data type DECIMAL to INT in CPI Column.
ALTER TABLE STUDENT_DETAIL
ALTER COLUMN CPI INT
- Delete Column City from the student_detail table.
ALTER TABLE STUDENT_DETAIL
DROP COLUMN CITY
- Rename Column Enrollment_No to ENO.
SP_RENAME 'STUDENT_DETAIL.ENROLLMENT','ENO'
- Change name of table student_detail to STUDENT_MASTER.
SP_RENAME 'STUDENT_DETAIL','STUDENT_MASTER'

DELETE, Truncate, Drop Operation

Part – A:

Use Deposit_Detail table (Altered table of DEPOSIT)

DEPOSIT_DETAIL	
Column_Name	Data Type
ANO	INT
CustomerName	VARCHAR(35)

BNAME	VARCHAR(50)
AMOUNT	INT
PINCODE	INT

1. Delete all the records of DEPOSIT_DETAIL table having amount less than and equals to 4000.
DELETE FROM DEPOSIT_DETAIL WHERE AMOUNT<=4000
2. Delete all the accounts CHANDI BRANCH.
DELETE FROM DEPOSIT_DETAIL WHERE BNAME='CHANDI'
3. Delete all the accounts having account number (ANO) is greater than 102 and less than 105.
DELETE FROM DEPOSIT_DETAIL WHERE ANO>102 AND ANO<105
4. Delete all the accounts whose branch is 'AJNI' or 'POWAI'
DELETE FROM DEPOSIT_DETAIL WHERE BNAME = 'AJNI' OR BNAME='POWAI'
5. Delete all the accounts whose account number is NULL.
DELETE FROM DEPOSIT_DETAIL WHERE ANO IS NULL
6. Delete all the remaining records using Delete command.
DELETE FROM DEPOSIT_DETAIL
7. Delete all the records of Deposit_Detail table. (Use **Truncate**)
TRUNCATE TABLE DEPOSIT_DETAIL
8. Remove Deposit_Detail table. (Use **Drop**)
DROP TABLE DEPOSIT_DETAIL

Part – B:

Create following table using query according to the definition.

EMPLOYEE_MASTER	
Column_Name	DataType
EmpNo	INT
EmpName	VARCHAR(25)
JoiningDate	DATETIME
Salary	DECIMAL (8,2)
City	VARCHAR(20)

Insert the following records in the EMPLOYEE_MASTER table.

EmpNo	EmpName	JoiningDate	Salary	City
101	Keyur	5-1-02	12000.00	Rajkot
102	Hardik	15-2-04	14000.00	Ahmedabad
103	Kajal	14-3-06	15000.00	Baroda
104	Bhoomi	23-6-05	12500.00	Ahmedabad
105	Harmit	15-2-04	14000.00	Rajkot
106	Mitesh	25-9-01	5000.00	Jamnagar
107	Meera	Null	7000.00	Morbi

108	Kishan	6-2-03	10000.00	NULL
-----	--------	--------	----------	------

From the above given tables perform the following queries (DELETE Operation):

1. Delete all the records of Employee_MASTER table having salary greater than and equals to 14000.
DELETE FROM EMPLOYEE_MASTER WHERE SALARY>=14000
2. Delete all the Employees who belongs to 'RAJKOT' city.
DELETE FROM EMPLOYEE_MASTER WHERE CITY='RAJKOT'
3. Delete all the Employees who joined after 1-1-2007.
DELETE FROM EMPLOYEE_MASTER WHERE JOININGDATE>'2007-01-01'
4. Delete the records of Employees whose joining date is null and Name is not null.
DELETE FROM EMPLOYEE_MASTER WHERE JOININGDATE IS NULL AND EMPNAME IS NOT NULL
5. Delete the records of Employees whose salary is 50% of 20000.
DELETE FROM EMPLOYEE_MASTER WHERE SALARY=10000
6. Delete the records of Employees whose City Name is not empty.
DELETE FROM EMPLOYEE_MASTER WHERE CITY<> NULL
7. Delete all the records of Employee_MASTER table. (Use **Truncate**)
TRUNCATE TABLE EMPLOYEE_MASTER
8. Remove Employee_MASTER table. (Use **Drop**)
DROP TABLE EMPLOYEE_MASTER

Part – C:

1. Summarize Delete, Truncate and Drop

Lab 3 Perform SQL queries for Like operator

Part – A:

Create following table using query according to the definition.

STUDENT	
Column_Name	DataType
StuID	INT
FirstName	VARCHAR(25)
LastName	VARCHAR(25)
Website	VARCHAR(50)
City	VARCHAR(25)
Address	VARCHAR(100)

Insert the following records in the STUDENT table.

StuID	FirstName	LastName	Website	City	Address
1011	Keyur	Patel	techonthenet.com	Rajkot	A-303 'Vasant Kunj', Rajkot
1022	Hardik	Shah	digminecraft.com	Ahmedabad	"Ram Krupa", Raiya Road
1033	Kajal	Trivedi	bigactivities.com	Baroda	Raj bhavan plot, near garden
1044	Bhoomi	Gajera	checkyourmath.com	Ahmedabad	"Jig's Home", Narol
1055	Harmit	Mitel	@me.darshan.com	Rajkot	B-55, Raj Residency
1066	Ashok	Jani	NULL	Baroda	A502, Club House Building

From the above given tables perform the following queries (LIKE Operation):

1. Display the name of students whose name starts with 'k'.
`SELECT FIRSTNAME FROM STUDENT WHERE FIRSTNAME LIKE 'K%'`
2. Display the name of students whose name consists of five characters.
`SELECT FIRSTNAME FROM STUDENT WHERE FIRSTNAME LIKE '_____'`
3. Retrieve the first name & last name of students whose city name ends with 'a' & contains six characters.
`SELECT FIRSTNAME, LASTNAME FROM STUDENT WHERE CITY LIKE '_____A'`
4. Display all the students whose last name ends with 'tel'.
`SELECT FIRSTNAME, LASTNAME FROM STUDENT WHERE LASTNAME LIKE '%TEL'`
5. Display all the students whose first name starts with 'ha' & ends with 't'.
`SELECT * FROM STUDENT WHERE FIRSTNAME LIKE 'HA%T'`
6. Display all the students whose first name starts with 'k' and third character is 'y'.
`SELECT * FROM STUDENT WHERE FIRSTNAME LIKE 'K_Y%'`
7. Display the name of students having no website and name consists of five characters.
`SELECT FIRSTNAME FROM STUDENT WHERE WEBSITE IS NULL AND FIRSTNAME LIKE '_____'`
8. Display all the students whose last name consist of 'jer'.
`SELECT * FROM STUDENT WHERE LASTNAME LIKE '%JER%'`
9. Display all the students whose city name starts with either 'r' or 'b'.
`SELECT * FROM STUDENT WHERE CITY LIKE '[R,B]%'`
10. Display all the name students having websites.
`SELECT * FROM STUDENT WHERE WEBSITE IS NOT NULL`
11. Display all the students whose name starts from alphabet A to H.
`SELECT * FROM STUDENT WHERE FIRSTNAME LIKE '[A-H]%'`
12. Display all the students whose name's second character is vowel.
`SELECT * FROM STUDENT WHERE FIRSTNAME LIKE '_[A,E,I,O,U]%'`
13. Display the name of students having no website and name consists of minimum five characters.
`SELECT FIRSTNAME FROM STUDENT WHERE WEBSITE IS NOT NULL AND FIRSTNAME LIKE '_____%'`
14. Display all the students whose last name starts with 'Pat'.
`SELECT * FROM STUDENT WHERE LASTNAME LIKE 'PAT%'`
15. Display all the students whose city name does not starts with 'b'.
`SELECT * FROM STUDENT WHERE CITY NOT LIKE 'B%'`

16. Display all the students whose student ID ends with digit.
`SELECT * FROM STUDENT WHERE STUID LIKE '%[0-9]'`
17. Display all the students whose address does not contain any digit.
`SELECT * FROM STUDENT WHERE ADDRESS NOT LIKE '%[0-9]%'`
18. Find students whose first name starts with 'B', last name ends with 'A', and their website contains either 'math' or 'science'. Ensure that their city does not start with 'B'.
`SELECT * FROM STUDENT
WHERE FIRSTNAME LIKE 'B%'
AND LASTNAME LIKE '%A'
AND WEBSITE LIKE '%[MATH,SCIENCE]%'
AND CITY NOT LIKE 'B%'`
19. Retrieve students who have 'Shah' in their last name and whose city ends with 'd'. Additionally, their website should be either null or contain 'com'.
`SELECT * FROM STUDENT
WHERE LASTNAME LIKE '%SHAH%'
AND CITY LIKE '%D'
AND (WEBSITE IS NULL OR WEBSITE LIKE '%COM%')`
20. Select students whose first and second character is a vowel. Their city should start with 'R' and they must have a website containing '.com'.
`SELECT * FROM STUDENT
WHERE FIRSTNAME LIKE '[A,E,I,O,U][A,E,I,O,U]%'
AND CITY LIKE 'R%'
AND WEBSITE LIKE '%.COM%'`

Part – B:

1. Display all the students whose name's second character is vowel and of and start with H.
`SELECT FIRSTNAME FROM STUDENT
WHERE FIRSTNAME LIKE '_[A,E,I,O,U]%'
AND FIRSTNAME LIKE '%H'`
2. Display all the students whose last name does not ends with 'a'.
`SELECT FIRSTNAME FROM STUDENT WHERE LASTNAME NOT LIKE '%A'`
3. Display all the students whose first name starts with consonant.
`SELECT FIRSTNAME FROM STUDENT WHERE FIRSTNAME NOT LIKE '[A,E,I,O,U]%'`
4. Retrieve student details whose first name starts with 'K', last name ends with 'tel', and either their website contains 'tech' or they live in a city starting with 'R'.
`SELECT * FROM STUDENT
WHERE FIRSTNAME LIKE 'K%'
AND LASTNAME LIKE '%TEL'
AND (WEBSITE LIKE '%TECH%' OR CITY LIKE 'R%')`
5. Retrieve students whose address contains a hyphen '-' and whose city starts with either 'R' or 'B'. They must have a website that ends with '.com' and their first name should not start with 'A'.

```
SELECT * FROM STUDENT
WHERE ADDRESS LIKE '%-%'
      AND CITY LIKE '[R,B]%'
      AND WEBSITE LIKE '%.COM'
      AND FIRSTNAME NOT LIKE 'A%'
```

Part – C:

1. Display all the students whose address contains single quote or double quote.

```
SELECT * FROM STUDENT
WHERE ADDRESS LIKE '%"%'
      OR ADDRESS LIKE '%"'
```

2. Find students whose city does not contain the letter 'S' and their address contains either single or double quotes. Their last name should start with 'P' and they must have a website that contains 'on'.

```
SELECT * FROM STUDENT
WHERE CITY LIKE '%S%'
      AND (ADDRESS LIKE '%"%' OR ADDRESS LIKE '% "')
      AND LASTNAME LIKE 'P%'
      AND WEBSITE LIKE '%ON%'
```

Lab 4

Perform SQL queries for Aggerate function and group by (without having)

Part – A:

Create table and inset records as per below.

EMP						
EID	EName	Department	Salary	JoiningDate	City	Gender
101	Rahul	Admin	56000	1-Jan-90	Rajkot	Male
102	Hardik	IT	18000	25-Sep-90	Ahmedabad	Male
103	Bhavin	HR	25000	14-May-91	Baroda	Male
104	Bhoomi	Admin	39000	8-Feb-91	Rajkot	Female
105	Rohit	IT	17000	23-Jul-90	Jamnagar	Male
106	Priya	IT	9000	18-Oct-90	Ahmedabad	Female
107	Bhoomi	HR	34000	25-Dec-91	Rajkot	Female

1. Display the Highest, Lowest, Label the columns Maximum, Minimum respectively.

```
SELECT MAX(SALARY) AS HIGHEST,
      MIN(SALARY) AS LOWEST FROM EMP
```

2. Display Total, and Average salary of all employees. Label the columns Total_Sal and Average_Sal, respectively.

```
SELECT SUM(SALARY) AS TOTAL_SAL,
      AVG(SALARY) AS AVERAGE_SAL FROM EMP
```

3. Find total number of employees of EMPLOYEE table.

```
SELECT COUNT(*) AS TOTAL_NO FROM EMP
```

4. Find highest salary from Rajkot city.

```
SELECT MAX(SALARY) AS HIGHEST
FROM EMP WHERE CITY='RAJKOT'
```

5. Give maximum salary from IT department.

```
SELECT MAX(SALARY) AS HIGHEST_SAL
FROM EMP WHERE DEPARTMENT='IT'
```

6. Count employee whose joining date is after 8-feb-91.

```
SELECT COUNT(*) AS TOTAL_NO
FROM EMP WHERE JOININGDATE>'8-FEB-91'
```

7. Display average salary of Admin department.

```
SELECT AVG(SALARY) AS AVERAGE_SAL
FROM EMP WHERE DEPARTMENT='ADMIN'
```

8. Display total salary of HR department.

```
SELECT SUM(SALARY) AS TOTAL_SAL
FROM EMP WHERE DEPARTMENT='HR'
```

9. Count total number of cities of employee without duplication.

```
SELECT COUNT(DISTINCT CITY) AS TOTAL_CITY FROM EMP
```

10. Count unique departments.

```
SELECT COUNT(DISTINCT DEPARTMENT) AS TOTAL_DEPT FROM EMP
```

11. Give minimum salary of employee who belongs to Ahmedabad.

```
SELECT MIN(SALARY) AS LOWEST
FROM EMP WHERE CITY='AHMEDABAD'
```

12. Find city wise highest salary.

```
SELECT CITY,MAX(SALARY) AS HIGHEST_SAL
FROM EMP GROUP BY CITY
```

13. Find department wise lowest salary.

```
SELECT DEPARTMENT,MIN(SALARY) AS LOWEST_SAL
FROM EMP
GROUP BY DEPARTMENT
```

14. Display city with the total number of employees belonging to each city.

```
SELECT CITY,COUNT(EID) AS TOTAL_NO
FROM EMP GROUP BY CITY
```

15. Give total salary of each department of EMP table.

```
SELECT DEPARTMENT,SUM(SALARY) AS TOTAL_SAL
FROM EMP GROUP BY DEPARTMENT
```

16. Give average salary of each department of EMP table without displaying the respective department name.

```
SELECT AVG(SALARY) AS AVG_SAL FROM EMP GROUP BY DEPARTMENT
```

17. Count the number of employees for each department in every city.

```
SELECT CITY,DEPARTMENT,COUNT(EID) AS EMP_COUNT
FROM EMP GROUP BY DEPARTMENT,CITY
```

18. Calculate the total salary distributed to male and female employees.

```
SELECT GENDER,SUM(SALARY) AS TOTAL_SALARY
FROM EMP GROUP BY GENDER
```

19. Give city wise maximum and minimum salary of female employees.

```
SELECT CITY,MAX(SALARY) AS MAX_SAL,MIN(SALARY) AS MIN_SAL
FROM EMP WHERE GENDER='FEMALE' GROUP BY CITY
```

20. Calculate department, city, and gender wise average salary.

```
SELECT DEPARTMENT,CITY,GENDER,AVG(SALARY) AS AVG_SAL
FROM EMP GROUP BY DEPARTMENT,CITY,GENDER
```

Part – B:

1. Count the number of employees living in Rajkot.

```
SELECT COUNT(*) AS TOTAL_NO
FROM EMP WHERE CITY='RAJKOT'
```

2. Display the difference between the highest and lowest salaries. Label the column DIFFERENCE.

```
SELECT MAX(SALARY) - MIN(SALARY) AS DIFFERENCE FROM EMP
```

3. Display the total number of employees hired before 1st January, 1991.

```
SELECT COUNT(*) AS TOTAL_EMP
FROM EMP WHERE JOINGINGDATE<'1-JANUARY-91'
```

Part – C:

1. Count the number of employees living in Rajkot or Baroda.

```
SELECT COUNT(*) AS TOTAL_EMP
FROM EMP WHERE CITY IN ('RAJKOT','BARODA')
```

2. Display the total number of employees hired before 1st January, 1991 in IT department.

```
SELECT COUNT(*) AS TOTAL_EMP
FROM EMP WHERE JOINGINGDATE<'1-JANUARY-91' AND DEPARTMENT='IT'
```

3. Find the Joining Date wise Total Salaries.

```
SELECT JOINGINGDATE,SUM(SALARY) AS TOTAL_SAL
FROM EMP GROUP BY JOINGINGDATE
```

4. Find the Maximum salary department & city wise in which city name starts with 'R'.

```
SELECT CITY,DEPARTMENT,MAX(SALARY) AS MAX_SAL
FROM EMP WHERE CITY LIKE 'R%' GROUP BY CITY,DEPARTMENT
```

Lab 5 Perform SQL queries for Group by with having and Order by

Region	Product	Sales_Amount	Year
North America	Watch	1500	2023

Table: SALES_DATA

Europe	Mobile	1200	2023
Asia	Watch	1800	2023
North America	TV	900	2024
Europe	Watch	2000	2024
Asia	Mobile	1000	2024
North America	Mobile	1600	2023
Europe	TV	1500	2023
Asia	TV	1100	2024
North America	Watch	1700	2024

Part – A:

- Display Total Sales Amount by Region.

```

SELECT
    REGION,
    SUM(SALES_AMOUNT) AS TOTAL_SALES
FROM SALES_DATA
GROUP BY REGION

```
- Display Average Sales Amount by Product.

```

SELECT
    PRODUCT,
    AVG(SALES_AMOUNT) AS ASAVG_SAL
FROM SALES_DATA
GROUP BY PRODUCT

```
- Display Maximum Sales Amount by Year

```

SELECT
    YEAR,
    MAX(SALES_AMOUNT) AS MAX_SAL
FROM SALES_DATA
GROUP BY YEAR

```
- Display Minimum Sales Amount by Region and Year.

```

SELECT
    REGION, YEAR,
    MIN(SALES_AMOUNT) AS MIN_SAL
FROM SALES_DATA
GROUP BY REGION, YEAR

```
- Count of Products Sold by Region

```

SELECT
    REGION,
    COUNT(PRODUCT) AS COUNT_PRO
FROM SALES_DATA
GROUP BY REGION

```
- Display Sales Amount by Year and Product

```

SELECT

```

```
YEAR,PRODUCT,
SUM(SALES_AMOUNT) AS TOTAL_SAL
FROM SALES_DATA
GROUP BY YEAR,PRODUCT
```

7. Display Regions with Total Sales Greater Than 5000.
SELECT

```
REGION,
SUM(SALES_AMOUNT)AS TOTAL_SAL
FROM SALES_DATA
GROUP BY REGION
HAVING SUM(SALES_AMOUNT)>5000
```

8. Display Products with Average Sales Less Than 10000.
SELECT

```
PRODUCT,
AVG(SALES_AMOUNT)AS AVG_SAL
FROM SALES_DATA
GROUP BY PRODUCT
HAVING AVG(SALES_AMOUNT)<10000
```

9. Display Years with Maximum Sales Exceeding 500
SELECT

```
YEAR,
MAX(SALES_AMOUNT)AS MAX_SAL
FROM SALES_DATA
GROUP BY YEAR
HAVING MAX(SALES_AMOUNT)>500
```

10. Display Regions with at Least 3 Distinct Products Sold.
SELECT

```
REGION,
COUNT(DISTINCT PRODUCT)AS DISTINCT_PRODUCT
FROM SALES_DATA
GROUP BY REGION
HAVING COUNT(DISTINCT PRODUCT)>=3
```

11. Display Years with Minimum Sales Less Than 1000
SELECT

```
YEAR,
MIN(SALES_AMOUNT)AS MIN_SAL
FROM SALES_DATA
GROUP BY YEAR
HAVING MIN(SALES_AMOUNT)<1000
```

12. Display Total Sales Amount by Region for Year 2023, Sorted by Total Amount


```
SELECT
    REGION,
    SUM(SALES_AMOUNT)AS TOTAL_SAL
FROM SALES_DATA
WHERE YEAR=2023
GROUP BY REGION
ORDER BY SUM(SALES_AMOUNT)
```

13. Find the Region Where 'Mobile' Had the Lowest Total Sales Across All Years.

```
SELECT
    TOP 1 REGION,YEAR,
    SUM(SALES_AMOUNT) AS TOTAL_SALES
FROM SALES_DATA
WHERE PRODUCT='MOBILE'
GROUP BY REGION,YEAR
ORDER BY TOTAL_SALES
```

14. Find the Product with the Highest Sales Across All Regions in 2023.

```
SELECT
    PRODUCT,
    MAX(SALES_AMOUNT)
FROM SALES_DATA
WHERE YEAR=2023
GROUP BY REGION
```

15. Find Regions Where 'TV' Sales in 2023 Were Greater Than 1000.

```
SELECT REGION,SUM(SALES_AMOUNT)
FROM SALES_DATA
WHERE PRODUCT = 'TV' AND YEAR = 2023
GROUP BY REGION
HAVING SUM(SALES_AMOUNT)>1000
```

Part – B:

1. Display Count of Orders by Year and Region, Sorted by Year and Region

```
SELECT
    YEAR,
    REGION,
    COUNT(*) AS COUNT_SALES
FROM SALES_DATA
GROUP BY YEAR,REGION
ORDER BY YEAR,REGION
```

2. Display Regions with Maximum Sales Amount Exceeding 1000 in Any Year, Sorted by Region

```
SELECT
    YEAR,
    REGION,
    MAX(SALES_AMOUNT) AS MAX_SAL
FROM SALES_DATA
GROUP BY YEAR,REGION
HAVING MAX(SALES_AMOUNT)>1000
ORDER BY REGION
```

3. Display Years with Total Sales Amount Less Than 10000, Sorted by Year Descending

```
SELECT  
    YEAR,  
    SUM(SALES_AMOUNT) AS TOTAL_SAL  
FROM SALES_DATA  
GROUP BY YEAR  
HAVING SUM(SALES_AMOUNT)<1000  
ORDER BY YEAR DESC
```

4. Display Top 3 Regions by Total Sales Amount in Year 2024

```
SELECT TOP 3 REGION,SUM(SALES_aMOUNT) AS TOTAL_SAL  
FROM SALES_DATA  
WHERE YEAR =2024  
GROUP BY REGION  
ORDER BY TOTAL_SAL
```

5. Find the Year with the Lowest Total Sales Across All Regions.

```
SELECT TOP 1 YEAR,SUM(SALES_AMOUNT) AS TOTAL_SAL  
FROM SALES_DATA  
GROUP BY YEAR  
ORDER BY TOTAL_SAL
```

Part – C:

1. Display Products with Average Sales Amount Between 1000 and 2000, Ordered by Product Name

```
SELECT  
    PRODUCT,  
    AVG(SALES_AMOUNT) AS AVG_SAL  
FROM SALES_DATA  
GROUP BY PRODUCT  
HAVING AVG(SALES_AMOUNT) BETWEEN 1000 AND 2000  
ORDER BY PRODUCT
```

2. Display Years with More Than 1 Orders from Each Region

```
SELECT  
    REGION,  
    YEAR,  
    COUNT(PRODUCT) AS ORDER_PRO  
FROM SALES_DATA  
GROUP BY REGION,YEAR  
HAVING COUNT(PRODUCT)>1
```

3. Display Regions with Average Sales Amount Above 1500 in Year 2023 sort by amount in descending.

```
SELECT  
    REGION,  
    AVG(SALES_AMOUNT) AS AVG_SALES  
FROM SALES_DATA  
WHERE YEAR=2023  
GROUP BY REGION  
HAVING AVG(SALES_aMOUNT)>1500  
ORDER BY SALES_AMOUNT DESC
```

	4. Find out region wise duplicate product. SELECT REGION,PRODUCT,COUNT(*) AS COUNT_PRODUCT FROM SALES_DATA GROUP BY REGION,PRODUCT HAVING COUNT(*) > 1; 5. Find out year wise duplicate product. SELECT YEAR,PRODUCT,COUNT(*) AS COUNT_PRODUCT FROM SALES_DATA GROUP BY YEAR,PRODUCT HAVING COUNT(*) >1
Lab 6	Implement SQL In-built functions (Math, String, and Date Functions)
	<u>Math functions</u> Part – A: 1. Display the result of 5 multiply by 30. SELECT (5*30); 2. Find out the absolute value of -25, 25, -50 and 50. SELECT ABS(-25),ABS(25),ABS(-50),ABS(50); 3. Find smallest integer value that is greater than or equal to 25.2, 25.7 and -25.2. SELECT CEILING(25.2),CEILING(25.7),CEILING(-25.2); 4. Find largest integer value that is smaller than or equal to 25.2, 25.7 and -25.2. SELECT FLOOR(25.2),FLOOR(25.7),FLOOR(-25.2) 5. Find out remainder of 5 divided 2 and 5 divided by 3. SELECT (5%2),(5%3); 6. Find out value of 3 raised to 2nd power and 4 raised 3rd power. SELECT POWER(3,2),POWER(4,3); 7. Find out the square root of 25, 30 and 50. SELECT SQRT(25),SQRT(30),SQRT(50); 8. Find out the square of 5, 15, and 25.

SELECT SQUARE(5),SQUARE(15),SQUARE(25)

9. Find out the value of PI.

SELECT PI();

10. Find out round value of 157.732 for 2, 0 and -2 decimal points.

SELECT ROUND(157.732,2),ROUND(157.732,0),ROUND(157.732,-2)

11. Find out exponential value of 2 and 3.

SELECT EXP(2),EXP(3)

12. Find out logarithm having base e of 10 and 2.

SELECT LOG(10),LOG(2)

13. Find out logarithm having base b having value 10 of 5 and 100.

SELECT LOG10(5),LOG10(100)

14. Find sine, cosine and tangent of 3.1415.

SELECT SIN(3.1415),COS(3.1415),TAN(3.1415)

15. Find sign of -25, 0 and 25.

SELECT SIGN(-25),SIGN(0),SIGN(25)

16. Generate random number using function.

SELECT RAND();

Part – B:

Create and insert the following records in the EMP_MASTER table.

EmpNo	EmpName	JoiningDate	Salary	Commission	City	Dept Code
101	Keyur	5-1-02	12000.00	4500	Rajkot	3@g
102	Hardik	15-2-04	14000.00	2500	Ahmedabad	3@
103	Kajal	14-3-06	15000.00	3000	Baroda	3-GD
104	Bhoomi	23-6-05	12500.00	1000	Ahmedabad	1A3D
105	Harmit	15-2-04	14000.00	2000	Rajkot	312A

1. Display the result of Salary plus Commission.

SELECT SALARY+COMMISSION AS TOTAL FROM EMP_MASTER

- Find smallest integer value that is greater than or equal to 55.2, 35.7 and -55.2.

```
SELECT CEILING(55.2),CEILING(35.7),CEILING(-55.2);
```

- Find largest integer value that is smaller than or equal to 55.2, 35.7 and -55.2.

```
SELECT FLOOR(55.2),FLOOR(35.7),FLOOR(-55.2);
```

- Find out remainder of 55 divided 2 and 55 divided by 3.

```
SELECT (55%2),(55%3)
```

- Find out value of 23 raised to 2nd power and 14 raised 3rd power.

```
SELECT POWER(23,2),POWER(14,3)
```

Part – C:

- Retrieve the details of employees whose total earnings (Salary + Commission) are greater than 15000.

```
SELECT * FROM EMP_MASTER  
WHERE SALARY+COMMISSION > 15000
```

- Find the details of employees whose commission is more than 25% of their salary.

```
SELECT * FROM EMP_MASTER WHERE COMMISSION > (0.25*SALARY)
```

- List the employees who joined before 2005 and whose total earnings (Salary + Commission) are greater than 15000.

```
SELECT * FROM EMP_MASTER WHERE JOININGDATE < '2005-01-01' AND (SALARY+COMMISSION)  
>15000
```

- Find employees whose total earnings (Salary + Commission) are at least double their salary.

```
SELECT * FROM EMP_MASTER WHERE (SALARY+COMMISSION) >= (2*SALARY)
```

String functions

Part – A:

- Find the length of following. (I) NULL (II) ' hello ' (III) Blank

```
SELECT LEN(NULL),LEN('HELLO'),LEN('');
```

- Display your name in lower & upper case.

```
SELECT LOWER('PUSHTI'),UPPER('Pushti')
```

- Display first three characters of your name.

SELECT LEFT('PUSHTI',3);

4. Display 3rd to 10th character of your name.

SELECT SUBSTRING('PUSHTI MARKANA',3,8);

5. Write a query to convert 'abc123efg' to 'abcXYZefg' & 'abcbcabcb' to 'ab5ab5ab5' using REPLACE.

SELECT REPLACE('abc123efg','123','XYZ'),REPLACE('abcbcabcb','c','5');

6. Write a query to display ASCII code for 'a','A','z','Z', 0, 9.

SELECT ASCII('a'),ASCII('A'),ASCII('z'),ASCII('Z'),ASCII('0'),ASCII('9')

7. Write a query to display character based on number 97, 65,122,90,48,57.

SELECT CHAR(97),CHAR(65),CHAR(122),CHAR(90),CHAR(48),CHAR(57);

8. Write a query to remove spaces from left of a given string 'hello world'.

SELECT LTRIM(' HELLO WORLD');

9. Write a query to remove spaces from right of a given string 'hello world'.

SELECT RTRIM('HELLO WORLD');

10. Write a query to display first 4 & Last 5 characters of 'SQL Server'.

SELECT LEFT('SQLSERVER',4),RIGHT('SQLSERVER',5)

11. Write a query to convert a string '1234.56' to number (Use cast and convert function).

SELECT CAST('1234.56'AS DECIMAL(6,2)),CONVERT(DECIMAL(6,2),'1234.56');

12. Write a query to convert a float 10.58 to integer (Use cast and convert function).

SELECT CAST(10.58 AS INT),CONVERT(INT,10.58);

13. Put 10 space before your name using function.

SELECT SPACE(10)+'PUSHTI';

14. Combine two strings using + sign as well as CONCAT ().

SELECT 'DARSHAN'+ 'UNIVERSITY',CONCAT('DARSHAN',' ','UNIVERSITY');

15. Find reverse of "Darshan".

SELECT REVERSE('DARSHAN');

16. Repeat your name 3 times.

SELECT REPLICATE('PUSHTI',3);

Part – B: Perform following queries on EMP_MASTER table.

1. Find the length of EMP Name and City columns.

SELECT LEN(EMPNAME),LEN(CITY) FROM EMP_MASTER;

2. Display EMP Name and City columns in lower & upper case.

SELECT LOWER(EMPNAME),LOWER(CITY),UPPER(EMPNAME),UPPER(CITY) FROM EMP_MASTER;

3. Display first three characters of EMP Name column.

SELECT LEFT(EMPNAME,3) FROM EMP_MASTER;

4. Display 3rd to 10th character of city column.

SELECT SUBSTRING(CITY,3,8) FROM EMP_MASTER;

5. Write a query to display first 4 & Last 5 characters of EMP Name column.

SELECT LEFT(EMPNAME,4),RIGHT(EMPNAME,5) FROM EMP_MASTER;

Part – C: Perform following queries on EMP_MASTER table.

1. Put 10 space before EMP Name using function.

SELECT SPACE(10)+EMPNAME FROM EMP_MASTER;

2. Combine EMP Name and city columns using + sign as well as CONCAT ().

SELECT EMPNAME+' '+CITY ,CONCAT(EMPNAME,' ',CITY) FROM EMP_MASTER;

3. Combine all columns using + sign as well as CONCAT ().

SELECT CAST(EMPNO AS varchar(20))+EMPNAME+CAST(JOININGDATE AS varchar(50))+CAST(SALARY AS varchar(50))+CAST(COMMISSION AS varchar(50))+CITY AS COMBINED_COLUMN FROM EMP_MASTER;

SELECT CONCAT_WS(' ',EMPNO,EMPNAME,JOININGDATE,SALARY,CITY) FROM EMP_MASTER;

4. Combine the result as < EMP Name > Lives in <City>.

```
SELECT EMPNAME +' Lives in '+CITY FROM EMP_MASTER;
```

5. Combine the result as 'EMP no of < EMP Name> is <EmpNo> .

```
SELECT 'EMP nO of '+EMPNAME+' is '+EMPNO FROM EMP_MASTER;
```

6. Retrieve the names of all employee where the third character of the Name is a vowel.

```
SELECT EMPNAME FROM EMP_MASTER
```

```
WHERE EMPNAME LIKE '__[A,E,I,O,U]%';
```

7. Concatenate the name and city of students who have a name that ends with the letter 'r' and a city that starts with 'R'.

```
SELECT EMPNAME+CITY FROM EMP_MASTER
```

```
WHERE EMPNAME LIKE '%r' AND CITY LIKE 'R%';
```

Date Functions

Part – A:

1. Write a query to display the current date & time. Label the column Today_Date.

```
SELECT GETDATE() AS TODAY_DATE;
```

2. Write a query to find new date after 365 day with reference to today.

```
SELECT DATEADD(D,365,GETDATE());
```

3. Display the current date in a format that appears as may 5 1994 12:00AM.

```
SELECT CONVERT(VARCHAR,GETDATE(),100);
```

4. Display the current date in a format that appears as 03 Jan 1995.

```
SELECT CONVERT(VARCHAR,GETDATE(),106);
```

5. Display the current date in a format that appears as Jan 04, 96.

```
SELECT CONVERT(VARCHAR,GETDATE(),107);
```

6. Write a query to find out total number of months between 31-Dec-08 and 31-Mar-09.

```
SELECT DATEDIFF(MONTH,'2008-12-31','2009-03-31');
```

7. Write a query to find out total number of hours between 25-Jan-12 7:00 and 26-Jan-12 10:30.

```
SELECT DATEDIFF(HOUR,'2012-01-25 7:00','2012-01-26 10:30');
```

8. Write a query to extract Day, Month, Year from given date 12-May-16.

```
SELECT DAY('2016-05-12'),MONTH('2016-05-12'),YEAR('2016-05-12');
```


9. Write a query that adds 5 years to current date.

```
SELECT DATEADD(YEAR,5,GETDATE());
```

10. Write a query to subtract 2 months from current date.

```
SELECT DATEADD(MONTH,-2,GETDATE());
```

11. Extract month from current date using datename () and datepart () function.

```
SELECT DATENAME(MONTH,GETDATE()),DATEPART(MONTH,GETDATE());
```

12. Write a query to find out last date of current month.

```
SELECT EOMONTH(GETDATE());
```

13. Calculate your age in years and months.

```
SELECT DATEDIFF(YEAR,'2005-05-19',GETDATE()) AS YEAR,DATEDIFF(MONTH,'2005-05-19',GETDATE())  
AS MONTH;
```

Part – B: Perform following queries on EMP_MASTER table.

1. Write a query to find new date after 365 days with reference to JoiningDate.

```
SELECT DATEADD(D,365,JOININGDATE) FROM EMP_MASTER;
```

2. Write a query to find out total number of months between JoiningDate and 31-Mar-09.

```
SELECT DATEDIFF(MONTH,JOININGDATE,'2009-03-31') FROM EMP_MASTER;
```

3. Write a query to find out total number of years between JoiningDate and 14-Sep-10.

```
SELECT DATEDIFF(YEAR,JOININGDATE,'2010-09-14') FROM EMP_MASTER;
```

Part – C: Perform following queries on EMP_MASTER table.

1. Write a query to extract Day, Month, Year from JoiningDate.

```
SELECT  
DATENAME(DAY,JOININGDATE),DATEPART(MONTH,JOININGDATE),DATENAME(YEAR,JOININGDATE)  
FROM EMP_MASTER;
```

2. Write a query that adds 5 years to JoiningDate.

```
SELECT DATEADD(YEAR,5,JOININGDATE) FROM EMP_MASTER;
```

3. Write a query to subtract 2 months from JoiningDate.

```
SELECT DATEADD(MONTH,-2,JOININGDATE) FROM EMP_MASTER;
```

4. Extract month from JoiningDate using datename () and datepart () function.

```
SELECT DATENAME(MONTH,JOININGDATE),DATEPART(MONTH,JOININGDATE) FROM EMP_MASTER;
```

5. Select employee who joined between the 1st and 15th of any month in any year.

```
SELECT * FROM EMP_MASTER  
WHERE DAY(JOININGDATE) BETWEEN 1 AND 15
```

6. Find employee whose JoiningDate is the last day of any month.

```
SELECT * FROM EMP_MASTER
```

WHERE DAY(JOININGDATE)= DAY(EOMONTH(JOININGDATE))

7. List employee whose JoiningDate is during a leap year.

SELECT * FROM EMP_MASTER

WHERE YEAR(JOININGDATE)%4=0

AND YEAR(JOININGDATE)%100!=0

OR YEAR(JOININGDATE)%400=0

Lab 7

Implement SQL Joins

Create below tables as per following data

STU_INFO		
Rno(PK)	Name	Branch
101	Raju	CE
102	Amit	CE
103	Sanjay	ME
104	Neha	EC
105	Meera	EE
106	Mahesh	ME

RESULT	
Rno(FK)	SPI
101	8.8
102	9.2
103	7.6
104	8.2
105	7.0
107	8.9

EMPLOYEE_MASTER		
EmployeeNo	Name	ManagerNo
E01	Tarun	NULL
E02	Rohan	E02
E03	Priya	E01
E04	Milan	E03
E05	Jay	E01
E06	Anjana	E04

Part – A:

1. Combine information from student and result table using cross join or Cartesian product.

SELECT * FROM Stu_Info

CROSS JOIN Result

2. Perform inner join on Student and Result tables.

SELECT Stu_Info.Rno, Stu_Info.Name, Stu_Info.Branch, Result.SPI FROM Stu_Info

INNER JOIN Result

ON Stu_Info.Rno = Result.Rno

3. Perform the left outer join on Student and Result tables.

SELECT Stu_Info.Rno, Stu_Info.Name, Stu_Info.Branch, Result.SPI FROM Stu_Info

LEFT OUTER JOIN Result

ON Stu_Info.Rno = Result.Rno

4. Perform the right outer join on Student and Result tables.

SELECT Stu_Info.Rno, Stu_Info.Name, Stu_Info.Branch, Result.SPI FROM Stu_Info

RIGHT OUTER JOIN Result

ON Stu_Info.Rno = Result.Rno

5. Perform the full outer join on Student and Result tables.

SELECT Stu_Info.Rno, Stu_Info.Name, Stu_Info.Branch, Result.SPI FROM Stu_Info

FULL OUTER JOIN Result

ON Stu_Info.Rno = Result.Rno

6. Display Rno, Name, Branch and SPI of all students.

SELECT Stu_Info.Rno, Stu_Info.Name, Stu_Info.Branch, Result.SPI FROM Stu_Info

LEFT OUTER JOIN Result

ON Stu_Info.Rno = Result.Rno

7. Display Rno, Name, Branch and SPI of CE branch's student only.

SELECT Stu_Info.Rno, Stu_Info.Name, Stu_Info.Branch, Result.SPI FROM Stu_Info

INNER JOIN Result

ON Stu_Info.Rno = Result.Rno

WHERE Stu_Info.Branch = 'CE'

8. Display Rno, Name, Branch and SPI of other than EC branch's student only.

SELECT Stu_Info.Rno, Stu_Info.Name, Stu_Info.Branch, Result.SPI FROM Stu_Info

INNER OUTER JOIN Result

ON Stu_Info.Rno = Result.Rno

WHERE Stu_Info.Branch != 'EC'

9. Display average result of each branch.

SELECT Stu_Info.Branch, AVG(Result.SPI) FROM Stu_Info

INNER JOIN Result

ON Stu_Info.Rno = Result.Rno

GROUP BY Stu_Info.Branch

10. Display average result of CE and ME branch.

SELECT Stu_Info.Branch, AVG(Result.SPI) AS Average_SPI FROM Stu_Info

INNER JOIN Result

ON Stu_Info.Rno = Result.Rno

WHERE Stu_Info.Branch IN ('CE', 'ME')

GROUP BY Stu_Info.Branch

11. Display Maximum and Minimum SPI of each branch.

SELECT Stu_Info.Branch, MIN(Result.SPI) AS Min_SPI, MAX(Result.SPI) AS Max_SPI FROM Stu_Info

INNER JOIN Result

ON Stu_Info.Rno = Result.Rno

GROUP BY Stu_Info.Branch

12. Display branch wise student's count in descending order.

SELECT Stu_Info.Branch, COUNT(Stu_Info.Rno) AS COUNT_STUDENT FROM Stu_Info

INNER JOIN Result

ON Stu_Info.Rno = Result.Rno

GROUP BY Stu_Info.Branch

ORDER BY Stu_Info.Branch DESC

Part – B:

1. Display average result of each branch and sort them in ascending order by SPI.

SELECT Stu_Info.Branch, AVG(Result.SPI) AS Average_SPI FROM Stu_Info

INNER JOIN Result

ON Stu_Info.Rno = Result.Rno

GROUP BY Stu_Info.Branch

ORDER BY Average_SPI ASC

2. Display highest SPI from each branch and sort them in descending order.

SELECT Stu_Info.Branch, MAX(Result.SPI) AS Highest_SPI FROM Stu_Info

INNER JOIN Result

ON Stu_Info.Rno = Result.Rno

GROUP BY Stu_Info.Branch

ORDER BY Highest_SPI DESC

Part – C:

1. Retrieve the names of employee along with their manager's name from the Employee table.

SELECT E1.Name AS EmployeeName, E2.Name AS ManagerName FROM EMPLOYEE_MASTER AS E1

LEFT JOIN EMPLOYEE_MASTER AS E2

ON E1.ManagerNo = E2.EmployeeNo

Lab 8

Implement Complex Joins

Create following table (Using Design Mode)

PERSON		
Column_Name		
PersonID	Int	Primary Key
PersonName	Varchar (100)	Not Null
DepartmentID	Int	Foreign Key, Null
Salary	Decimal (8,2)	Not Null
JoiningDate	Datetime	Not Null
City	Varchar (100)	Not Null

DEPT		
Column_Name	Data Type	Constraints
DepartmentID	Int	Primary Key
DepartmentName	Varchar (100)	Not Null, Unique
DepartmentCode	Varchar (50)	Not Null, Unique
Location	Varchar (50)	Not Null

PersonID	PersonName	DepartmentID	Salary	JoiningDate	City
101	Rahul Tripathi	2	56000	01-01-2000	Rajkot
102	Hardik Pandya	3	18000	25-09-2001	Ahmedabad
103	Bhavin Kanani	4	25000	14-05-2000	Baroda
104	Bhoomi Vaishnav	1	39000	08-02-2005	Rajkot
105	Rohit Topiya	2	17000	23-07-2001	Jamnagar
106	Priya Menpara	NULL	9000	18-10-2000	Ahmedabad
107	Neha Sharma	2	34000	25-12-2002	Rajkot
108	Nayan Goswami	3	25000	01-07-2001	Rajkot
109	Mehul Bhundiya	4	13500	09-01-2005	Baroda
110	Mohit Maru	5	14000	25-05-2000	Jamnagar

DepartmentID	DepartmentName	DepartmentCode	Location
1	Admin	Adm	A-Block
2	Computer	CE	C-Block
3	Civil	CI	G-Block
4	Electrical	EE	E-Block
5	Mechanical	ME	B-Block

From the above given table perform the following queries:

Part – A:

1. Combine information from Person and Department table using cross join or Cartesian product.

SELECT * FROM Person

CROSS JOIN Dept

2. Find all persons with their department name
SELECT P.PersonName, D.DepartmentName FROM Person P
INNER JOIN Dept D
ON P.DepartmentID=D.DepartmentID
3. Find all persons with their department name & code.
SELECT P.PersonName, D.DepartmentName, D.DepartmentCode FROM Person P
INNER JOIN Dept D
ON P.DepartmentID=D.DepartmentID
4. Find all persons with their department code and location.
SELECT Person.PersonName, D.DepartmentCode,D.LocationFROM Person
INNER JOIN Dept D
ON Person.DepartmentID=D.DepartmentID
5. Find the detail of the person who belongs to Mechanical department.
SELECT P.PersonName, P.City, P.Salary, P.JoiningDate, D.DepartmentName, D.Location FROM Person P
INNER JOIN Dept D
ON P.DepartmentID=D.DepartmentID
WHERE D.DepartmentName='Mechanical'
6. Final person's name, department code and salary who lives in Ahmedabad city.
SELECT P.PersonName, D.DepartmentCode,P.SalaryFROM Person P
LEFT OUTER JOIN Dept D
ON P.DepartmentID=D.DepartmentID
WHERE P.City='Ahmedabad'
7. Find the person's name whose department is in C-Block.
SELECT P.PersonName FROM Person P
INNER JOIN Dept D
ON P.DepartmentID=D.DepartmentID
WHERE D.Location='C-Block'
8. Retrieve person name, salary & department name who belongs to Jamnagar city.
SELECT P.PersonName, P.Salary, D.DepartmentName FROM Person P
LEFT OUTER JOIN Dept D
ON P.DepartmentID=D.DepartmentID
WHERE P.City='Jamnagar'
9. Retrieve person's detail who joined the Civil department after 1-Aug-2001.
SELECT P.PersonName, P.City, P.Salary, P.JoiningDate ,D.DepartmentNameFROM Person P
LEFT OUTER JOIN Dept D
ON P.DepartmentID=D.DepartmentID

WHERE P.JoiningDate>'1-Aug-2001' and D.DepartmentName='Civil'

10. Display all the person's name with the department whose joining date difference with the current date is more than 365 days.

```
SELECT P.PersonName, D.DepartmentName, P.JoiningDate FROM Person P
INNER JOIN Dept D
ON P.DepartmentID=D.DepartmentID
WHERE DATEDIFF(DAY,P.JoiningDate,GETDATE())>365
```

11. Find department wise person counts.

```
SELECT D.DepartmentName, COUNT(P.PersonID) AS 'Person_count'FROM Person P
INNER JOIN Dept D
ON P.DepartmentID=D.DepartmentID
GROUP BY D.DepartmentName
```

12. Give department wise maximum & minimum salary with department name.

```
SELECT D.DepartmentName, MAX(P.Salary) "Max Salary", MIN(P.Salary) "Min Salary" FROM Person P
INNER JOIN Dept D
ON P.DepartmentID=D.DepartmentID
GROUP BY D.DepartmentName
```

13. Find city wise total, average, maximum and minimum salary.

```
SELECT P.City, Max(P.Salary) as MaxSalary, MIN(P.Salary) as MinSalary, AVG(P.Salary) as
AvgSalary,SUM(P.Salary) as TotalSalary FROM Person P
GROUP BY P.City
```

14. Find the average salary of a person who belongs to Ahmedabad city.

```
SELECT AVG(P.Salary) as AvgSalary, P.City FROM Person P
GROUP BY P.City
HAVING P.City='Ahmedabad'
```

15. Produce Output Like: <PersonName> lives in <City> and works in <DepartmentName> Department. (In single column)

```
SELECT P.PersonName + ' lives in ' + P.City + ' and works in ' + D.DepartmentName + ' Department'
FROM Person P
INNER JOIN Dept D
ON P.DepartmentID=D.DepartmentID
```

Part – B:

1. Produce Output Like: <PersonName> earns <Salary> from <DepartmentName> department monthly. (In single column)

```
SELECT P.PersonName + ' earns ' + CAST(Salary as varchar) + ' from Department ' + D.DepartmentName
```

+ ' monthly ' FROM Person P
INNER JOIN Dept D
ON P.DepartmentID=D.DepartmentID

2. Find city & department wise total, average & maximum salaries.
SELECT P.City, D.DepartmentName, Max(P.Salary) as MaxSalary, MIN(P.Salary) as MinSalary,
AVG(P.Salary) as AvgSalary, SUM(P.Salary) as TotalSalary FROM Person P
LEFT OUTER JOIN Dept D
ON P.DepartmentID=D.DepartmentID
GROUP BY P.City, D.DepartmentName
3. Find all persons who do not belong to any department.
SELECT P.PersonName
FROM Person P
WHERE P.DepartmentID is NULL
4. Find all departments whose total salary is exceeding 100000.
SELECT D.DepartmentName, SUM(P.Salary) "Total Dept Salary" FROM Person P
INNER JOIN Dept D
ON P.DepartmentID=D.DepartmentID
GROUP BY D.DepartmentName
HAVING SUM(P.Salary)>100000

Part – C:

1. List all departments who have no person.
SELECT D.DepartmentName FROM Person P
FULL OUTER JOIN Dept D
ON P.DepartmentID=D.DepartmentID
GROUP BY D.DepartmentName
HAVING COUNT (P.DepartmentID) =0
2. List out department names in which more than two persons are working.
SELECT D.DepartmentName, COUNT(*) as PersonCount FROM Person P
INNER JOIN Dept D
ON P.DepartmentID=D.DepartmentID
GROUP BY D.DepartmentName
HAVING COUNT(P.DepartmentID)>2
--OR--
SELECT D.DEPARTMENTNAME FROM PERSON P
INNER JOIN Dept D
ON P.DEPARTMENTID = D.DEPARTMENTID
GROUP BY D.DEPARTMENTNAME
HAVING COUNT(P.PERSONID) > 2;
3. Give a 10% increment in the computer department employee's salary. (Use Update)
UPDATE Person SET Person.Salary=(P.Salary+(P.Salary*10)/100) FROM Person P
INNER JOIN Dept D

	ON P.DepartmentID=D.DepartmentID WHERE D.DepartmentName='Computer'																																																											
Lab 9	Perform SQL queries for Subqueries																																																											
	<table><caption>STUDENT_DATA</caption><thead><tr><th>Rno</th><th>Name</th><th>City</th><th>DID</th></tr></thead><tbody><tr><td>101</td><td>Raju</td><td>Rajkot</td><td>10</td></tr><tr><td>102</td><td>Amit</td><td>Ahmedabad</td><td>20</td></tr><tr><td>103</td><td>Sanjay</td><td>Baroda</td><td>40</td></tr><tr><td>104</td><td>Neha</td><td>Rajkot</td><td>20</td></tr><tr><td>105</td><td>Meera</td><td>Ahmedabad</td><td>30</td></tr><tr><td>106</td><td>Mahesh</td><td>Baroda</td><td>10</td></tr></tbody></table><table><caption>DEPARTMENT</caption><thead><tr><th>DID</th><th>DName</th></tr></thead><tbody><tr><td>10</td><td>Computer</td></tr><tr><td>20</td><td>Electrical</td></tr><tr><td>30</td><td>Mechanical</td></tr><tr><td>40</td><td>Civil</td></tr></tbody></table><table><caption>ACADEMIC</caption><thead><tr><th>RNO</th><th>SPI</th><th>BKLOG</th></tr></thead><tbody><tr><td>101</td><td>8.8</td><td>0</td></tr><tr><td>102</td><td>9.2</td><td>2</td></tr><tr><td>103</td><td>7.6</td><td>1</td></tr><tr><td>104</td><td>8.2</td><td>4</td></tr><tr><td>105</td><td>7.0</td><td>2</td></tr><tr><td>106</td><td>8.9</td><td>3</td></tr></tbody></table> Part – A: - Display details of students who are from computer department. SELECT * FROM Stu_Data WHERE DID = (SELECT DID FROM Department WHERE DNAME = 'COMPUTER') - Displays name of students whose SPI is more than 8. SELECT NAME FROM Stu_Data WHERE RNO IN (SELECT RNO FROM Academic WHERE SPI > 8) - Display details of students of computer department who belongs to Rajkot city. SELECT * FROM Stu_Data WHERE CITY = 'RAJKOT' AND DID = (SELECT DID FROM Department WHERE DNAME = 'COMPUTER') - Find total number of students of electrical department. SELECT COUNT(RNO) FROM Stu_Data WHERE DID = (SELECT DID FROM Department WHERE DNAME = 'ELECTRICAL') - Display name of student who is having maximum SPI. SELECT NAME FROM Stu_Data WHERE RNO = (SELECT RNO FROM Academic WHERE SPI = (SELECT MAX(SPI) FROM Academic)) - Display details of students having more than 1 backlog. SELECT * FROM Stu_Data WHERE RNO IN (SELECT RNO FROM Academic WHERE BKLOG > 1) Part – B: - Display name of students who are either from computer department or from mechanical department. SELECT NAME FROM Stu_Data WHERE DID IN (SELECT DID FROM Department WHERE DNAME IN ('COMPUTER' 'MECHANICAL'))	Rno	Name	City	DID	101	Raju	Rajkot	10	102	Amit	Ahmedabad	20	103	Sanjay	Baroda	40	104	Neha	Rajkot	20	105	Meera	Ahmedabad	30	106	Mahesh	Baroda	10	DID	DName	10	Computer	20	Electrical	30	Mechanical	40	Civil	RNO	SPI	BKLOG	101	8.8	0	102	9.2	2	103	7.6	1	104	8.2	4	105	7.0	2	106	8.9	3
Rno	Name	City	DID																																																									
101	Raju	Rajkot	10																																																									
102	Amit	Ahmedabad	20																																																									
103	Sanjay	Baroda	40																																																									
104	Neha	Rajkot	20																																																									
105	Meera	Ahmedabad	30																																																									
106	Mahesh	Baroda	10																																																									
DID	DName																																																											
10	Computer																																																											
20	Electrical																																																											
30	Mechanical																																																											
40	Civil																																																											
RNO	SPI	BKLOG																																																										
101	8.8	0																																																										
102	9.2	2																																																										
103	7.6	1																																																										
104	8.2	4																																																										
105	7.0	2																																																										
106	8.9	3																																																										

2. Display name of students who are in same department as 102 studying in.

```
SELECT NAME FROM Stu_Data
WHERE DID IN ( SELECT DID FROM Stu_Data
WHERE RNO = 102)
```

Part – C:

1. Display name of students whose SPI is more than 9 and who is from electrical department.

```
SELECT NAME FROM Stu_Data
WHERE RNO IN ( SELECT RNO FROM Academic
WHERE SPI > 9)
AND DID IN ( SELECT DID FROM Department
WHERE DNAME = 'ELECTRICAL')
```

2. Display name of student who is having second highest SPI.

```
SELECT NAME FROM Stu_Data
WHERE RNO = (SELECT RNO FROM ACADEMIC
WHERE SPI = (SELECT MAX(SPI)
FROM ACADEMIC
WHERE SPI < (SELECT MAX(SPI)
FROM ACADEMIC)));
--OR--
SELECT NAME FROM Stu_Data
WHERE RNO=(SELECT RNO FROM ACADEMIC
WHERE SPI=( SELECT TOP 1 SPI
FROM (SELECT DISTINCT TOP 2 SPI
FROM ACADEMIC
ORDER BY SPI DESC) AS TEMP
ORDER BY SPI ASC) );
```

3. Display city names whose students SPI is 9.2

```
SELECT DISTINCT City FROM Stu_Data
WHERE Rno IN ( SELECT Rno FROM Academic
WHERE SPI = 9.2)
```

4. Find the names of students who have more than the average number of backlogs across all students.

```
SELECT NAME FROM Stu_Data
WHERE RNO IN (SELECT RNO FROM ACADEMIC
WHERE Bklog > (SELECT AVG(Bklog)
FROM ACADEMIC ));
```

5. Display the names of students who are in the same department as the student with the highest SPI.

```
SELECT NAME FROM Stu_Data
WHERE DID =( SELECT DID FROM Stu_Data
WHERE Rno = (SELECT RNO FROM Academic
WHERE SPI=(SELECT MAX(SPI)
FROM Academic)))
```

Lab 10

Implement Stored Procedure

Person

Column_Name	DataType	Constraints
PersonID	Int	Primary Key, Auto Increment
FirstName	Varchar (100)	Not Null
LastName	Varchar (100)	Not Null

Salary	Decimal (8,2)	Not Null
JoiningDate	Datetime	Not Null
DepartmentID	Int	Foreign Key, Null
DesignationID	Int	Foreign Key, Null

Department		
Column_Name	Data Type	Constraints
DepartmentID	Int	Primary Key
DepartmentName	Varchar (100)	Not Null, Unique

Designation		
Column_Name	Data Type	Constraints
DesignationID	Int	Primary Key
DesignationName	Varchar (100)	Not Null, Unique

PersonID	FirstName	LastName	Salary	JoiningDate	DepartmentID	DesignationID
101	Rahul	Anshu	56000	01-01-1990	1	12
102	Hardik	Hinsu	18000	25-09-1990	2	11
103	Bhavin	Kamani	25000	14-05-1991	NULL	11
104	Bhoomi	Patel	39000	20-02-2014	1	13
105	Rohit	Rajgor	17000	23-07-1990	2	15
106	Priya	Mehta	25000	18-10-1990	2	NULL
107	Neha	Trivedi	18000	20-02-2014	3	15

DepartmentID	DepartmentName
1	Admin
2	IT
3	HR
4	Account

DesignationID	DesignationName
11	Jobber
12	Welder
13	Clerk
14	Manager
15	CEO

From the above given tables create Stored Procedures:

Part – A

1. Department, Designation & Person Table's INSERT, UPDATE & DELETE Procedures.

-- Department INSERT Procedure

CREATE PROCEDURE PR_Department_Insert

@DepartmentID INT,

@DepartmentName VARCHAR(100)

AS

BEGIN

INSERT INTO Department (DepartmentID, DepartmentName)

VALUES (@DepartmentID, @DepartmentName);

END;

-- Department UPDATE Procedure

CREATE PROCEDURE PR_Department_Update

@DepartmentID INT,

@DepartmentName VARCHAR(100)

AS



```
BEGIN
    UPDATE Department
    SET DepartmentName = @DepartmentName
    WHERE DepartmentID = @DepartmentID;
END;
-- Department DELETE Procedure
CREATE PROCEDURE PR_Department_Delete
    @DepartmentID INT
AS
BEGIN
    DELETE FROM Department
    WHERE DepartmentID = @DepartmentID;
END
-- Designation INSERT Procedure
CREATE PROCEDURE PR_Designation_Insert
    @DesignationID INT,
    @DesignationName VARCHAR(100)
AS
BEGIN
    INSERT INTO Designation (DesignationID, DesignationName)
    VALUES (@DesignationID, @DesignationName);
END
-- Designation UPDATE Procedure
CREATE PROCEDURE PR_Designation_Update
    @DesignationID INT,
    @DesignationName VARCHAR(100)
AS
BEGIN
    UPDATE Designation
    SET DesignationName = @DesignationName
    WHERE DesignationID = @DesignationID;
END
-- Designation DELETE Procedure
CREATE PROCEDURE PR_Designation_Delete
    @DesignationID INT
AS
BEGIN
    DELETE FROM Designation
    WHERE DesignationID = @DesignationID;
END
-- Person INSERT Procedure
CREATE PROCEDURE PR_Person_Insert
    @FirstName VARCHAR(100),
    @LastName VARCHAR(100),
    @Salary DECIMAL(8, 2),
    @JoiningDate DATETIME,
    @DepartmentID INT = NULL,
```

```
@DesignationID INT = NULL
AS
BEGIN
    INSERT INTO Person (FirstName, LastName, Salary, JoiningDate, DepartmentID, DesignationID)
    VALUES (@FirstName, @LastName, @Salary, @JoiningDate, @DepartmentID, @DesignationID);
END
-- Person UPDATE Procedure
CREATE PROCEDURE PR_Person_Update
    @PersonID INT,
    @FirstName VARCHAR(100),
    @LastName VARCHAR(100),
    @Salary DECIMAL(8, 2),
    @JoiningDate DATETIME,
    @DepartmentID INT = NULL,
    @DesignationID INT = NULL
AS
BEGIN
    UPDATE Person
    SET FirstName = @FirstName, LastName = @LastName, Salary = @Salary, JoiningDate =
@JoiningDate,
        DepartmentID = @DepartmentID, DesignationID = @DesignationID
    WHERE PersonID = @PersonID;
END;
-- Person DELETE Procedure
CREATE PROCEDURE PR_Person_Delete
    @PersonID INT
AS
BEGIN
    DELETE FROM Person
    WHERE PersonID = @PersonID;
END
```

2. Department, Designation & Person Table's SELECTBYPRIMARYKEY

```
-- Department SELECTBYPRIMARYKEY Procedure
CREATE PROCEDURE PR_Department_SelectByPrimaryKey
    @DepartmentID INT
AS
BEGIN
    SELECT * FROM Department
    WHERE DepartmentID = @DepartmentID;
END

-- Designation SELECTBYPRIMARYKEY Procedure
CREATE PROCEDURE PR_Designation_SelectByPrimaryKey
    @DesignationID INT
AS
BEGIN
```

```
SELECT * FROM Designation
WHERE DesignationID = @DesignationID;
END
-- Person SELECTBYPRIMARYKEY Procedure
CREATE PROCEDURE PR_Person_SelectByPrimarykey
    @PersonID INT
AS
BEGIN
    SELECT P.*, D.DepartmentName, G.DesignationName
    FROM Person P
    LEFT JOIN Department D ON P.DepartmentID = D.DepartmentID
    LEFT JOIN Designation G ON P.DesignationID = G.DesignationID
    WHERE P.PersonID = @PersonID;
END
```

3. Department, Designation & Person Table's (If foreign key is available then do write join and take columns on select list)

```
-- Department SelectAllWithDetails Procedure
CREATE PROCEDURE PR_Department_SelectAllWithDetails
AS
BEGIN
    SELECT * FROM Department
END
```

```
-- Designation SelectAllWithDetails Procedure
CREATE PROCEDURE PR_Designation_SelectAllWithDetails
AS
BEGIN
    SELECT * FROM Designation
END
```

```
-- Person SelectAllWithDetails Procedure
CREATE PROCEDURE PR_Person_SelectAllWithDetails
AS
BEGIN
    SELECT P.PersonID, P.FirstName, P.LastName, P.Salary, P.JoiningDate, D.DepartmentName,
    G.DesignationName
    FROM Person P
    LEFT JOIN Department D ON P.DepartmentID = D.DepartmentID
    LEFT JOIN Designation G ON P.DesignationID = G.DesignationID;
END
```

4. Create a Procedure that shows details of the first 3 persons.

```
CREATE PROCEDURE PR_Person_ShowFirstThree
AS
BEGIN
    SELECT TOP 3 * FROM Person;
```

END

Part – B

1. Create a Proc that takes the dept name as input and returns a table with all workers working in that dept.

```
CREATE or ALTER PROCEDURE PR_Person_GetWorkersByDepartment
    @DepartmentName VARCHAR(100)
```

AS

BEGIN

```
    SELECT P.*
    FROM Person P
    JOIN Department D ON P.DepartmentID = D.DepartmentID
    WHERE D.DepartmentName = @DepartmentName;
```

END

2. Create Procedure that takes department name & designation name as input and returns a table with worker's first name, salary, joining date & department name.

```
CREATE or ALTER PROCEDURE PR_Person_GetWorkersByDeptAndDesig
    @DepartmentName VARCHAR(100),
    @DesignationName VARCHAR(100)
```

AS

BEGIN

```
    SELECT P.FirstName, P.Salary, P.JoiningDate, D.DepartmentName
    FROM Pe P
    JOIN Department D ON P.DepartmentID = D.DepartmentID
    JOIN Designation G ON P.DesignationID = G.DesignationID
    WHERE D.DepartmentName = @DepartmentName AND G.DesignationName = @DesignationName;
```

END

3. Create a Procedure that takes the first name as an input parameter and display all the details of the worker with their department & designation name.

```
CREATE or ALTER PROCEDURE PR_Person_GetWorkerDetailsByFirstName
    @FirstName VARCHAR(100)
```

AS

BEGIN

```
    SELECT P.*, D.DepartmentName, G.DesignationName
    FROM Pe P
    LEFT JOIN Department D ON P.DepartmentID = D.DepartmentID
    LEFT JOIN Designation G ON P.DesignationID = G.DesignationID
    WHERE P.FirstName = @FirstName;
```

END

4. Create Procedure which displays department wise maximum, minimum & total salaries.

```
CREATE or ALTER PROCEDURE PR_Department_GetSalaryStats
```

AS

BEGIN

```
    SELECT D.DepartmentName, MAX(P.Salary) AS MaxSalary, MIN(P.Salary) AS MinSalary,
```

```
SUM(P.Salary) AS TotalSalary
FROM Person P
JOIN Department D ON P.DepartmentID = D.DepartmentID
GROUP BY D.DepartmentName;
END
```

5. Create Procedure which displays designation wise average & total salaries.

```
CREATE or ALTER PROCEDURE PR_Designation_GetSalaryStats
AS
BEGIN
    SELECT G.DesignationName, AVG(P.Salary) AS AvgSalary, SUM(P.Salary) AS TotalSalary
    FROM Pe P
    JOIN Designation G ON P.DesignationID = G.DesignationID
    GROUP BY G.DesignationName;
END
```

Part – C

1. Create Procedure that Accepts Department Name and Returns Person Count.

```
CREATE or ALTER PROCEDURE PR_Department_GetPersonCount
    @DepartmentName VARCHAR(100)
AS
BEGIN
    SELECT COUNT(*) AS PersonCount
    FROM Pe P
    JOIN Department D ON P.DepartmentID = D.DepartmentID
    WHERE D.DepartmentName = @DepartmentName;
END
```

2. Create a procedure that takes a salary value as input and returns all workers with a salary greater than input salary value along with their department and designation details.

```
CREATE or ALTER PROCEDURE PR_Person_GetWorkersWithSalaryAbove
    @Salary DECIMAL(8,2)
AS
BEGIN
    SELECT P.*, D.DepartmentName, G.DesignationName
    FROM Pe P
    INNER JOIN Department D ON P.DepartmentID = D.DepartmentID
    INNER JOIN Designation G ON P.DesignationID = G.DesignationID
    WHERE P.Salary > @Salary
END
```

3. Create a procedure to find the department(s) with the highest total salary among all departments.

```
CREATE or ALTER PROCEDURE PR_Department_GetHighestTotalSalary
AS
BEGIN
    SELECT TOP 1 D.DepartmentName, SUM(P.Salary) AS TotalSalary
```

```
FROM Person P
JOIN Department D ON P.DepartmentID = D.DepartmentID
GROUP BY D.DepartmentName
ORDER BY TotalSalary DESC;
END
```

4. Create a procedure that takes a designation name as input and returns a list of all workers under that designation who joined within the last 10 years, along with their department.

```
CREATE or ALTER PROCEDURE PR_Designation_GetRecentWorkers
    @DesignationName VARCHAR(100)
AS
BEGIN
    SELECT P.*, D.DepartmentName
    FROM Pe P
    JOIN Designation G ON P.DesignationID = G.DesignationID
    JOIN Department D ON P.DepartmentID = D.DepartmentID
    WHERE G.DesignationName = @DesignationName AND DATEDIFF(YEAR, P.JoiningDate, GETDATE()) <
    10;
END
```

5. Create a procedure to list the number of workers in each department who do not have a designation assigned.

```
CREATE or ALTER PROCEDURE PR_Department_GetWorkersWithoutDesignation
AS
BEGIN
    SELECT D.DepartmentName, COUNT(P.PersonID) AS WorkerCount
    FROM Pe P
    JOIN Department D ON P.DepartmentID = D.DepartmentID
    WHERE P.DesignationID IS NULL
    GROUP BY D.DepartmentName;
END
```

6. Create a procedure to retrieve the details of workers in departments where the average salary is above 12000.

```
CREATE or ALTER PROCEDURE PR_Department_GetHighAvgSalaryWorkers
AS
BEGIN
    SELECT P.*, D.DepartmentName
    FROM Person P
    JOIN Department D ON P.DepartmentID = D.DepartmentID
    WHERE D.DepartmentID IN (
        SELECT DepartmentID
        FROM Person
        GROUP BY DepartmentID
        HAVING AVG(Salary) > 12000
    );
END
```


Lab 11	Implement Advanced Stored Procedure						
	Departments						
	ColumnName	DataType	AN NN	Remarks			
	DepartmentID	Int	NN	Primary Key			
	DepartmentName	Varchar(100)	NN				
	ManagerID	Int	NN				
	Location	Varchar(100)	NN				
	Employee						
	Column Name	Data Type	AN NN	Remarks			
	EmployeeID	Int	NN	Primary Key			
	FirstName	Varchar(100)	NN				
	LastName	Varchar(100)	NN				
	DoB	Datetime	NN				
	Gender	Varchar(50)	NN				
	HireDate	Datetime	NN				
	DeptID	Int	NN	Foreign Key			
	Salary	Decimal(10,2)	NN				
	Projects						
	ColumnName	DataType	AN NN	Remarks			
	ProjectID	Int	NN	Primary Key			
	ProjectName	Varchar(100)	NN				
	StartDate	Datetime	NN				
	EndDate	Datetime	NN				
	DepartmentID	Int	NN	Foreign Key			
	DepartmentID DepartmentName ManagerID Location						
	1	IT	101	New York			
2	HR	102	San Francisco				
3	Finance	103	Los Angeles				
4	Admin	104	Chicago				
5	Marketing	105	Miami				
EmployeeID FirstName LastName DoB Gender HireDate DeptID Salary							
101	John	Doe	1985-04-12	Male	2010-06-15	1	75000.00
102	Jane	Smith	1990-08-24	Female	2015-03-10	2	60000.00
103	Robert	Brown	1982-12-05	Male	2008-09-25	3	82000.00
104	Emily	Davis	1988-11-11	Female	2012-07-18	4	58000.00
105	Michael	Wilson	1992-02-02	Male	2018-11-30	5	67000.00
ProjectID ProjectName StartDate EndDate DepartmentID							
201	Project Alpha	2022-01-01	2022-12-31	1			
202	Project Beta	2023-03-15	2024-03-14	2			
203	Project Gamma	2021-06-01	2022-05-31	3			
204	Project Delta	2020-10-10	2021-10-09	4			

205

Project Epsilon

2024-04-01

2025-03-31

5

Part – A

1. Create Stored Procedure for Employee table As User enters either First Name or Last Name and based on this you must give EmployeeID, DOB, Gender & Hiredate.

```
CREATE PROCEDURE PR_GetEmployeeByName (
    @FirstName VARCHAR(100) = NULL,
    @LastName VARCHAR(100) = NULL
)
AS
BEGIN
    SELECT EmployeeID, DoB, Gender, HireDate FROM Employee
    WHERE FirstName = @FirstName OR LastName = @LastName
END
```

2. Create a Procedure that will accept Department Name and based on that gives employees list who belongs to that department.

```
CREATE PROCEDURE PR_GetEmployeesByDepartment (
    @DepartmentName VARCHAR(100)
)
AS
BEGIN
    SELECT
        [dbo].[Employee].[EmployeeID],
        [dbo].[Employee].[FirstName],
        [dbo].[Employee].[LastName],
        [dbo].[Employee].[Salary]
    FROM
        [dbo].[Employee]
    INNER JOIN
        [dbo].[Departments]
    ON
        [dbo].[Employee].[DepartmentID] = [dbo].[Departments].[DepartmentID]
    WHERE
        [dbo].[Departments].[DepartmentName] = @DepartmentName
END
```

3. Create a Procedure that accepts Project Name & Department Name and based on that you must give all the project related details.

```
CREATE PROCEDURE PR_GetProjectDetails (
    @ProjectName VARCHAR(100),
    @DepartmentName VARCHAR(100)
)
AS
BEGIN
    SELECT
        [dbo].[Projects].[ProjectID],
        [dbo].[Projects].[ProjectName],
        [dbo].[Projects].[StartDate],
        [dbo].[Projects].[EndDate],
        [dbo].[Departments].[DepartmentName]
    FROM
        [dbo].[Projects]
```

```

INNER JOIN
[dbo].[Departments]
ON
[dbo].[Projects].[DepartmentID] = [dbo].[Departments].[DepartmentID]
WHERE
[dbo].[Projects].ProjectName = @ProjectName AND
[dbo].[Departments].[DepartmentName] = @DepartmentName
END

```

4. Create a procedure that will accepts any integer and if salary is between provided integer, then those employee list comes in output.

```

CREATE PROCEDURE PR_GetEmployeesBySalary (
    @MinSalary INT,
    @MaxSalary INT
)
AS
BEGIN
SELECT EmployeeID, FirstName, LastName, Salary
FROM Employee
WHERE Salary BETWEEN @MinSalary AND @MaxSalary
END

```

5. Create a Procedure that will accepts a date and gives all the employees who all are hired on that date.

```

CREATE PROCEDURE PR_GetEmployeesByHireDate (
    @HireDate DATETIME
)
AS
BEGIN SELECT EmployeeID, FirstName, LastName, HireDate
FROM Employee
WHERE HireDate = @HireDate;
END

```

Part – B

1. Create a Procedure that accepts Gender's first letter only and based on that employee details will be served.

```

CREATE PROCEDURE PR_GetEmployeesByGenderInitial (
    @GenderInitial CHAR(1)
)
AS
BEGIN
SELECT EmployeeID, FirstName, LastName, Gender
FROM Employee
WHERE LEFT(Gender, 1) = @GenderInitial
END

```

2. Create a Procedure that accepts First Name or Department Name as input and based on that employee data will come.

```

CREATE PROCEDURE PR_GetEmployeesByFirstNameOrDepartment (
    @FirstName VARCHAR(100) = NULL,

```

```

        @DepartmentName VARCHAR(100) = NULL
    )
    AS
    BEGIN
        SELECT E.EmployeeID, E.FirstName, E.LastName, D.DepartmentName FROM
Employee as E
        INNER JOIN Departments as D
        ON
        E.DepartmentID = D.DepartmentID
        WHERE E.FirstName = @FirstName
        OR
        D.DepartmentName = @DepartmentName
    END

```

3. Create a procedure that will accepts location, if user enters a location any characters, then he/she will get all the departments with all data.

```

CREATE PROCEDURE PR_GetDepartmentsByLocation (
        @Location VARCHAR(100)
    )
    AS
    BEGIN
        SELECT DepartmentID, DepartmentName, ManagerID, Location
        FROM Departments
        WHERE Location LIKE '%' + @Location + '%'
    END

```

Part – C

1. Create a procedure that will accepts From Date & To Date and based on that he/she will retrieve Project related data.

```

CREATE PROCEDURE PR_GetProjectsByDateRange (
        @FromDate DATETIME,
        @ToDate DATETIME
    )
    AS
    BEGIN
        SELECT ProjectID, ProjectName, StartDate, EndDate, DepartmentID
        FROM Projects
        WHERE StartDate >= @FromDate AND EndDate <= @ToDate;
    END

```

2. Create a procedure in which user will enter project name & location and based on that you must provide all data with Department Name, Manager Name with Project Name & Starting Ending Dates.

```

CREATE PROCEDURE PR_GetProjectAndDepartmentDetails (
        @ProjectName VARCHAR(100),
        @Location VARCHAR(100)
    )
    AS
    BEGIN
        SELECT P.ProjectID, P.ProjectName, P.StartDate, P.EndDate, D.DepartmentName,
D.ManagerID,
D.Location FROM Projects P
        INNER JOIN Departments D
        ON P.DepartmentID = D.DepartmentID
        WHERE P.ProjectName = @ProjectName AND D.Location LIKE '%' + @Location + '%'
    END

```

	END
Lab 12	Implement UDF
	Note: for Table valued function use tables of Lab-2 Part – A - Write a function to print "hello world". <pre>CREATE FUNCTION fn_HelloWorld() RETURNS VARCHAR(25) AS BEGIN RETURN 'hello world'; END</pre> - Write a function which returns addition of two numbers. <pre>CREATE FUNCTION fn_AdditionOfTwoNumbers (@num1 INT, @num2 INT) RETURNS INT AS BEGIN return @num1+@num2 END</pre> - Write a function to check whether the given number is ODD or EVEN. <pre>CREATE FUNCTION fn_NumberIsOddOrEven (@num INT) RETURNS VARCHAR(10) AS BEGIN DECLARE @ans AS VARCHAR(10); IF @num % 2 = 0 SET @ans = 'EVEN'; ELSE SET @ans = 'ODD'; RETURN @ans; END</pre> - Write a function which returns a table with details of a person whose first name starts with B. <pre>CREATE FUNCTION fn_DetailsOfPersonStartsWithB() RETURNS TABLE AS RETURN (SELECT * FROM Pe WHERE FirstName LIKE 'B%')</pre> - Write a function which returns a table with unique first names from the person table. <pre>CREATE FUNCTION fn_TableWithUniqueFirstName() RETURNS TABLE AS</pre>

```
RETURN ( SELECT DISTINCT FirstName FROM Pe)
```

6. Write a function to print number from 1 to N. (Using while loop)

```
CREATE FUNCTION fn_1toNnumbers  
( @num int)  
RETURNS VARCHAR(500)  
AS  
BEGIN  
    DECLARE @ans AS VARCHAR(MAX)  
    SET @ans=' '  
    DECLARE @i AS INT  
    SET @i=1  
    WHILE @i<=@num  
    BEGIN  
        SET @ans=@ans+CAST(@i AS VARCHAR)+' '  
        SET @i=@i+1  
    END  
    RETURN @ans  
END
```

7. Write a function to find the factorial of a given integer.

```
CREATE FUNCTION fn_FindFactorial  
( @num INT )  
RETURNS INT  
AS  
BEGIN  
    DECLARE @ans as INT  
    SET @ans=1;  
    DECLARE @i AS INT;  
    SET @i=1;  
    WHILE @i<=@num  
    BEGIN  
        SET @ans = @ans*@i;  
        SET @i = @i + 1;  
    END  
    RETURN @ans  
END
```

Part – B

1. Write a function to compare two integers and return the comparison result. (Using Case statement)

```
CREATE FUNCTION fn_CompareIntegers(@Num1 INT, @Num2 INT)  
RETURNS VARCHAR(20)  
AS  
BEGIN  
    RETURN CASE  
        WHEN @Num1 > @Num2 THEN 'First is greater'  
        WHEN @Num1 < @Num2 THEN 'Second is greater'  
        ELSE 'Both are equal'
```

END

END

2. Write a function to print the sum of even numbers between 1 to 20.

```
CREATE FUNCTION fn_SumOfEvens()
```

```
RETURNS INT
```

```
AS
```

```
BEGIN
```

```
    DECLARE @ans INT = 0;
```

```
    DECLARE @i INT = 2;
```

```
    WHILE @i <= 20
```

```
    BEGIN
```

```
        SET @ans = @ans + @i;
```

```
        SET @i = @i + 2;
```

```
    END
```

```
    RETURN @ans
```

```
END
```

3. Write a function that checks if a given string is a palindrome

```
CREATE FUNCTION fn_IsPalindrome(@Text VARCHAR(100))
```

```
RETURNS VARCHAR(20)
```

```
AS
```

```
BEGIN
```

```
    DECLARE @ReversedText VARCHAR(100) = REVERSE(@Text);
```

```
    RETURN CASE
```

```
        WHEN @Text = @ReversedText THEN 'Palindrome'
```

```
        ELSE 'Not a Palindrome'
```

```
    END
```

```
END
```

Part – C

1. Write a function to check whether a given number is prime or not.

```
CREATE FUNCTION fn_IsPrime(@num INT)
```

```
RETURNS VARCHAR(20)
```

```
AS
```

```
BEGIN
```

```
    DECLARE @i INT = 2;
```

```
    IF @num <= 1
```

```
        RETURN 'Not Prime';
```

```
    WHILE @i <= @num/2
```

```
    BEGIN
```

```
        IF @num % @i = 0
```

```
            RETURN 'Not Prime';
```

```
SET @i = @i + 1;
END
```

```
RETURN 'Prime'
END
```

- Write a function which accepts two parameters start date & end date, and returns a difference in days.

```
CREATE FUNCTION fn_DateDifference(@StartDate DATE, @EndDate DATE)
    RETURNS INT
    AS
    BEGIN
        RETURN DATEDIFF(DAY, @StartDate, @EndDate);
    END
```
- Write a function which accepts two parameters year & month in integer and returns total days each year.

```
CREATE FUNCTION fn_TotalDaysInMonth(@Year INT, @Month INT)
    RETURNS INT
    AS
    BEGIN
        RETURN DAY(EOMONTH(DATEFROMPARTS(@Year, @Month, 1)));
    END
```
- Write a function which accepts departmentID as a parameter & returns a detail of the persons.

```
CREATE FUNCTION fn_GetPersonsByDepartmentID(@DepartmentID INT)
    RETURNS TABLE
    AS
    RETURN
    (
        SELECT * FROM Person
        WHERE DepartmentID = @DepartmentID
    )
```
- Write a function that returns a table with details of all persons who joined after 1-1-1991.

```
CREATE FUNCTION fn_GetPersonsJoinedAfter1991()
    RETURNS TABLE
    AS
    RETURN
    (
        SELECT * FROM Person
        WHERE JoiningDate > '1991-01-01'
    )
```

Lab 13

Implement Cursor

Products

Column_Name	Data Type	Constraints
Product_id	Int	Primary Key
Product_Name	Varchar (250)	Not Null

Price	Decimal (10,2)	Not Null
-------	----------------	----------

Products		
Product_id	Product_Name	Price
1	Smartphone	35000
2	Laptop	65000
3	Headphones	5500
4	Television	85000
5	Gaming Console	32000

From the above given tables perform the following queries:

Part - A

1. Create a cursor Product_Cursor to fetch all the rows from a products table.

```
DECLARE @PID INT,@PNAME VARCHAR(250),@PRICE DECIMAL(10,2)
```

```
DECLARE PRODUCT_CURSOR CURSOR
```

```
FOR
```

```
SELECT Product_id,Product_Name,Price FROM Products
```

```
OPEN PRODUCT_CURSOR
```

```
FETCH NEXT FROM PRODUCT_CURSOR
```

```
INTO @PID,@PNAME,@PRICE
```

```
WHILE @@FETCH_STATUS=0
```

```
BEGIN
```

```
PRINT CAST(@PID AS VARCHAR)+'-'+@PNAME+'-'+ CAST(@PRICE AS VARCHAR)
```

```
FETCH NEXT FROM PRODUCT_CURSOR
```

```
INTO @PID,@PNAME,@PRICE
```

```
END
```

```
CLOSE PRODUCT_CURSOR
```

```
DEALLOCATE PRODUCT_CURSOR
```

2. Create a cursor Product_Cursor_Fetch to fetch the records in form of ProductID_ProductName.
(Example: 1_Smartphone)

```
DECLARE @P_ID INT,@P_NAME VARCHAR(250)
```

```
DECLARE PRODUCT_CURSOR_FETCH CURSOR
```

```
FOR
```

```
SELECT Product_id,Product_Name FROM Products
```

```
OPEN PRODUCT_CURSOR_FETCH
```

```
FETCH NEXT FROM PRODUCT_CURSOR_FETCH
```

```
INTO @P_ID,@P_NAME
```

```
WHILE @@FETCH_STATUS=0
BEGIN
    PRINT CAST(@P_ID AS VARCHAR)+'_'+@P_NAME

    FETCH NEXT FROM PRODUCT_CURSOR_FETCH
    INTO @P_ID,@P_NAME
END

CLOSE PRODUCT_CURSOR_FETCH
DEALLOCATE PRODUCT_CURSOR_FETCH
```

3. Create a Cursor to Find and Display Products Above Price 30,000.
DECLARE @PRID INT,@PRNAME VARCHAR(205) ,@PR DECIMAL(10,2)

```
DECLARE PRODUCT_CURSOR_PRICE CURSOR
FOR
    SELECT Product_id,Product_Name,Price FROM Products
    WHERE Price>30000
OPEN PRODUCT_CURSOR_PRICE

FETCH NEXT FROM PRODUCT_CURSOR_PRICE
INTO @PRID,@PRNAME,@PR

WHILE @@FETCH_STATUS=0
BEGIN
    PRINT CAST(@PRID AS VARCHAR)+'-'+@PRNAME+'-'+ CAST(@PR AS VARCHAR)

    FETCH NEXT FROM PRODUCT_CURSOR_PRICE
    INTO @PRID,@PRNAME,@PR
END

CLOSE PRODUCT_CURSOR_PRICE
DEALLOCATE PRODUCT_CURSOR_PRICE
```

4. Create a cursor Product_CursorDelete that deletes all the data from the Products table.
DECLARE @PR_ID INT

```
DECLARE PRODUCT_CURSOR_DELETE CURSOR
FOR
    SELECT Product_id FROM Products
OPEN PRODUCT_CURSOR_DELETE

FETCH NEXT FROM PRODUCT_CURSOR_DELETE
INTO @PR_ID
```

```
WHILE @@FETCH_STATUS=0
BEGIN
    DELETE FROM Products
    WHERE Product_id= @PR_ID

    FETCH NEXT FROM PRODUCT_CURSOR_DELETE
    INTO @PR_ID
END

CLOSE PRODUCT_CURSOR_DELETE
DEALLOCATE PRODUCT_CURSOR_DELETE
```

Part – B

1. Create a cursor Product_CursorUpdate that retrieves all the data from the products table and increases the price by 10%.

```
DECLARE @PRO_ID INT
DECLARE PRODUCT_CURSORUPDATE CURSOR
FOR
    SELECT Product_id FROM Products
OPEN PRODUCT_CURSORUPDATE

FETCH NEXT FROM PRODUCT_CURSORUPDATE
INTO @PRO_ID

WHILE @@FETCH_STATUS=0
BEGIN
    UPDATE Products
    SET Price = Price + (Price*10)/100
    WHERE Product_id = @PRO_ID

    FETCH NEXT FROM PRODUCT_CURSORUPDATE
    INTO @PRO_ID
END

CLOSE PRODUCT_CURSORUPDATE
DEALLOCATE PRODUCT_CURSORUPDATE
```

2. Create a Cursor to Rounds the price of each product to the nearest whole number.

```
DECLARE @PROD_ID INT

DECLARE PRODUCT_CURSOR_ROUND CURSOR
FOR
    SELECT Product_id FROM Products
OPEN PRODUCT_CURSOR_ROUND

FETCH NEXT FROM PRODUCT_CURSOR_ROUND
INTO @PROD_ID
```

```
WHILE @@FETCH_STATUS=0
BEGIN
    UPDATE Products
    SET Price = ROUND(Price,0)
    WHERE Product_id = @PROD_ID

    FETCH NEXT FROM PRODUCT_CURSOR_ROUND
    INTO @PROD_ID
END

CLOSE PRODUCT_CURSOR_ROUND
DEALLOCATE PRODUCT_CURSOR_ROUND
```

Part – C

1. Create a cursor to insert details of Products into the NewProducts table if the product is "Laptop" (Note: Create NewProducts table first with same fields as Products table)

```
DECLARE @PRODUCT_ID INT ,@PRODUCT_NAME VARCHAR(50),@NEWPRICE DECIMAL(10,2)
DECLARE NEWPRODUCT_INSERT CURSOR
FOR
    SELECT Product_id,Product_Name,Price FROM Products
OPEN NEWPRODUCT_INSERT
FETCH NEXT FROM NEWPRODUCT_INSERT
INTO @PRODUCT_ID,@PRODUCT_NAME,@NEWPRICE

WHILE @@FETCH_STATUS=0
BEGIN
    IF(@PRODUCT_NAME='Laptop')
        INSERT INTO NewProducts VALUES (@PRODUCT_ID,@PRODUCT_NAME,@NEWPRICE)

    FETCH NEXT FROM NEWPRODUCT_INSERT
    INTO @PRODUCT_ID,@PRODUCT_NAME,@NEWPRICE
END
```

```
CLOSE NEWPRODUCT_INSERT
DEALLOCATE NEWPRODUCT_INSERT
```

2. Create a Cursor to Archive High-Price Products in a New Table (ArchivedProducts), Moves products with a price above 50000 to an archive table, removing them from the original Products table.

```
DECLARE @PRODU_ID INT ,@PRODU_NAME VARCHAR(50),@ARPRICE DECIMAL(10,2)

DECLARE ARPRODUCT_INSERT CURSOR
FOR
    SELECT Product_id,Product_Name,Price FROM Products
OPEN ARPRODUCT_INSERT
FETCH NEXT FROM ARPRODUCT_INSERT
INTO @PRODU_ID,@PRODU_NAME,@ARPRICE
```

```

WHILE @@FETCH_STATUS=0
BEGIN
    IF @ARPRICE>=50000
    BEGIN
        INSERT INTO ArchivedProducts VALUES (@PRODU_ID,@PRODU_NAME,@ARPRICE)
        DELETE FROM Products
        WHERE Product_id=@PRODU_ID
    END

    FETCH NEXT FROM ARPRODUCT_INSERT
    INTO @PRODU_ID,@PRODU_NAME,@ARPRICE
END

CLOSE ARPRODUCT_INSERT
DEALLOCATE ARPRODUCT_INSERT
    
```

Lab 14 **Implement Trigger**

Create following table (Using Design Mode)

PersonInfo		
Column_Name	DataType	Constraints
PersonID	Int	Primary Key
PersonName	Varchar (100)	Not Null
Salary	Decimal (8,2)	Not Null
JoiningDate	Datetime	Allow Null
City	Varchar (100)	Not Null
Age	Int	Allow Null
BirthDate	Datetime	Not Null

PersonLog		
Column_Name	DataType	Constraints
PLogID	Int	Primary Key, Auto increment
PersonID	Int	Not Null
PersonName	Varchar (250)	Not Null
Operation	Varchar (50)	Not Null
UpdateDate	Datetime	Not Null

From the above given tables perform the following queries:

Part – A

1. Create a trigger that fires on INSERT, UPDATE and DELETE operation on the PersonInfo table to display a message "Record is Affected."

```

CREATE OR ALTER TRIGGER TR_INS_DEL_UPD
ON PersonInfo
AFTER INSERT,UPDATE,DELETE
AS
BEGIN
    
```

```
PRINT 'RECORD IS AFFECTED'
```

```
END
```

2. Create a trigger that fires on INSERT, UPDATE and DELETE operation on the PersonInfo table. For that, log all operations performed on the person table into PersonLog.

```
-----INSERT-----
```

```
CREATE OR ALTER TRIGGER TR_INSERT_LOG
```

```
ON PersonInfo
```

```
AFTER INSERT
```

```
AS
```

```
BEGIN
```

```
    DECLARE @PID INT,@PNAME VARCHAR(250)
```

```
    SELECT @PID = PersonID FROM inserted
```

```
    SELECT @PNAME=PersonName FROM inserted
```

```
    INSERT INTO PersonLog VALUES(@PID,@PNAME,'INSERT',GETDATE())
```

```
END
```

```
-----UPDATE-----
```

```
CREATE OR ALTER TRIGGER TR_UPDATE_LOG
```

```
ON PersonInfo
```

```
AFTER UPDATE
```

```
AS
```

```
BEGIN
```

```
    DECLARE @PID INT,@PNAME VARCHAR(250)
```

```
    SELECT @PID = PersonID FROM inserted
```

```
    SELECT @PNAME=PersonName FROM inserted
```

```
    INSERT INTO PersonLog VALUES(@PID,@PNAME,'UPDATE',GETDATE())
```

```
END
```

```
-----DELETE-----
```

```
CREATE OR ALTER TRIGGER TR_DELETED_LOG
```

```
ON PersonInfo
```

```
AFTER DELETE
```

```
AS
```

```
BEGIN
```

```
    DECLARE @PID INT,@PNAME VARCHAR(250)
```

```
    SELECT @PID = PersonID FROM deleted
```

```
    SELECT @PNAME=PersonName FROM deleted
```

```
    INSERT INTO PersonLog VALUES(@PID,@PNAME,'DELETE',GETDATE())
```

END

3. Create an INSTEAD OF trigger that fires on INSERT, UPDATE and DELETE operation on the PersonInfo table. For that, log all operations performed on the person table into PersonLog.

-----INSERT-----

CREATE OR ALTER TRIGGER TR_INS_INSERT_LOG

ON PersonInfo

INSTEAD OF INSERT

AS

BEGIN

DECLARE @PID INT,@PNAME VARCHAR(250)

SELECT @PID = PersonID FROM inserted

SELECT @PNAME=PersonName FROM inserted

INSERT INTO PersonLog VALUES(@PID,@PNAME,'INSERT',GETDATE())

END

-----UPDATE-----

CREATE OR ALTER TRIGGER TR_INS_UPDATE_LOG

ON PersonInfo

INSTEAD OF UPDATE

AS

BEGIN

DECLARE @PID INT,@PNAME VARCHAR(250)

SELECT @PID = PersonID FROM inserted

SELECT @PNAME=PersonName FROM inserted

INSERT INTO PersonLog VALUES(@PID,@PNAME,'UPDATE',GETDATE())

END

-----DALETE-----

CREATE OR ALTER TRIGGER TR_INS_DELETE_LOG

ON PersonInfo

INSTEAD OF DELETE

AS

BEGIN

DECLARE @PID INT,@PNAME VARCHAR(250)

SELECT @PID = PersonID FROM deleted

SELECT @PNAME=PersonName FROM deleted

INSERT INTO PersonLog VALUES(@PID,@PNAME,'UPDATE',GETDATE())

END

4. Create a trigger that fires on INSERT operation on the PersonInfo table to convert person name into uppercase whenever the record is inserted.

```
CREATE OR ALTER TRIGGER TR_INSERT_UPPER_LOG
```

```
ON PersonInfo
```

```
AFTER INSERT
```

```
AS
```

```
BEGIN
```

```
    DECLARE @PID INT,@PNAME VARCHAR(250)
```

```
    SELECT @PID = PersonID FROM inserted
```

```
    SELECT @PNAME=PersonName FROM inserted
```

```
    INSERT INTO PersonLog VALUES(@PID,UPPER(@PNAME),'INSERT',GETDATE())
```

END

5. Create trigger that prevent duplicate entries of person name on PersonInfo table.

```
CREATE OR ALTER Trigger tr_prevent_duplicate
```

```
on PersonInfo
```

```
INSTEAD OF INSERT
```

```
AS
```

```
BEGIN
```

```
    INSERT INTO PersonInfo(PersonID,PersonName,Salary,JoiningDate,City,Age,BirthDate)
```

```
    SELECT * FROM inserted
```

```
    WHERE PersonName NOT IN(SELECT PersonName FROM PersonInfo)
```

END

6. Create trigger that prevent Age below 18 years.

```
CREATE OR ALTER TRIGGER TR_PREVENT_AGE
```

```
ON PersonInfo
```

```
INSTEAD OF INSERT
```

```
AS
```

```
BEGIN
```

```
    INSERT INTO PersonInfo(PersonID,PersonName,Salary,JoiningDate,City,Age,BirthDate)
```

```
    SELECT * FROM INSERTED
```

```
    WHERE AGE>=18
```

END

Part – B

1. Create a trigger that fires on INSERT operation on person table, which calculates the age and update that age in Person table.

```
CREATE OR ALTER TRIGGER TR_CALCULATE_AGE
```

```
ON PersonInfo
```

```
AFTER INSERT
```


AS

BEGIN

DECLARE @PID INT,@DOB DATETIME,@AGE INT

SELECT @PID=PersonID ,@DOB= BirthDate FROM inserted
SET @AGE=FLOOR(DATEDIFF(DAY,@DOB, GETDATE())) / 365)

UPDATE PersonInfo
SET Age=@AGE WHERE PersonID=@PID

END

2. Create a Trigger to Limit Salary Decrease by a 10%.

CREATE OR ALTER TRIGGER TR_SALARY_LIMIT
ON PersonInfo
AFTER UPDATE

AS

BEGIN

DECLARE @OldSalary DECIMAL(8,2), @NewSalary DECIMAL(8,2),@PID INT

SELECT @PID=PersonID FROM inserted
SELECT @OldSalary = Salary FROM deleted WHERE PersonID=@PID
SELECT @NewSalary = Salary FROM inserted WHERE PersonID=@PID

IF @NewSalary < @OldSalary * 0.9

BEGIN

PRINT 'Salary decrease cannot be more than 10%'

UPDATE PersonInfo SET Salary = @OldSalary WHERE PersonID= (SELECT PersonID FROM
inserted)

END

END

Part – C

1. Create Trigger to Automatically Update JoiningDate to Current Date on INSERT if JoiningDate is NULL during an INSERT.

CREATE OR ALTER TRIGGER TR_UPDATE_DATE
ON PERSONINFO
INSTEAD OF INSERT

AS

BEGIN

DECLARE @PID INT,@JDATE DATETIME
SELECT @PID = PersonID FROM inserted
SELECT @JDATE= JoiningDate FROM inserted

INSERT INTO PersonInfo(PersonID,PersonName,Salary,JoiningDate,City,Age,BirthDate)

SELECT * FROM inserted

IF (@JDATE IS NULL)

BEGIN

UPDATE PersonInfo SET JoiningDate=GETDATE() WHERE PersonID=@PID

	END END 2. Create DELETE trigger on PersonLog table, when we delete any record of PersonLog table it prints 'Record deleted successfully from PersonLog'. CREATE OR ALTER TRIGGER TR_DELETE_PERSONLOG ON PersonLog AFTER DELETE AS BEGIN PRINT 'Record deleted successfully from PersonLog' END																																													
Lab 15	Implement Advanced Trigger																																													
	AFTER Trigger <table><tr><th colspan="3">EmployeeDetails</th></tr><tr><th>Column_Name</th><th>DataType</th><th>Constraints</th></tr><tr><td>EmployeeID</td><td>Int</td><td>Primary Key</td></tr><tr><td>EmployeeName</td><td>Varchar(100)</td><td>Not Null</td></tr><tr><td>ContactNo</td><td>Varchar(100)</td><td>Not Null</td></tr><tr><td>Department</td><td>Varchar(100)</td><td>Not Null</td></tr><tr><td>Salary</td><td>Decimal(10,2)</td><td>Not Null</td></tr><tr><td>Joining Date</td><td>DateTime</td><td>Allow Null</td></tr></table> <table><tr><th colspan="3">EmployeeLogs</th></tr><tr><th>Column_Name</th><th>DataType</th><th>Constraints</th></tr><tr><td>LogID</td><td>Int</td><td>Primary Key, Autoincrement</td></tr><tr><td>EmployeeID</td><td>Int</td><td>Not Null</td></tr><tr><td>EmployeeName</td><td>Varchar(100)</td><td>Not Null</td></tr><tr><td>ActionPerformed</td><td>Varchar(100)</td><td>Not Null</td></tr><tr><td>ActionDate</td><td>DateTime</td><td>Not Null</td></tr></table> Part – A 1. Create a trigger that fires AFTER INSERT, UPDATE, and DELETE operations on the EmployeeDetails table to display the message "Employee record affected." CREATE OR ALTER TRIGGER TR_INSERT_MSG ON EMPLOYEEDETAILS AFTER INSERT,UPDATE,DELETE AS BEGIN PRINT 'Employee record affected' END 2. Create a trigger that fires AFTER INSERT, UPDATE, and DELETE operations on the EmployeeDetails table to log all operations into the EmployeeLog table. -----INSERT----- CREATE OR ALTER TRIGGER TR_INSERT_LOG	EmployeeDetails			Column_Name	DataType	Constraints	EmployeeID	Int	Primary Key	EmployeeName	Varchar(100)	Not Null	ContactNo	Varchar(100)	Not Null	Department	Varchar(100)	Not Null	Salary	Decimal(10,2)	Not Null	Joining Date	DateTime	Allow Null	EmployeeLogs			Column_Name	DataType	Constraints	LogID	Int	Primary Key, Autoincrement	EmployeeID	Int	Not Null	EmployeeName	Varchar(100)	Not Null	ActionPerformed	Varchar(100)	Not Null	ActionDate	DateTime	Not Null
EmployeeDetails																																														
Column_Name	DataType	Constraints																																												
EmployeeID	Int	Primary Key																																												
EmployeeName	Varchar(100)	Not Null																																												
ContactNo	Varchar(100)	Not Null																																												
Department	Varchar(100)	Not Null																																												
Salary	Decimal(10,2)	Not Null																																												
Joining Date	DateTime	Allow Null																																												
EmployeeLogs																																														
Column_Name	DataType	Constraints																																												
LogID	Int	Primary Key, Autoincrement																																												
EmployeeID	Int	Not Null																																												
EmployeeName	Varchar(100)	Not Null																																												
ActionPerformed	Varchar(100)	Not Null																																												
ActionDate	DateTime	Not Null																																												

```
ON EMPLOYEEDETAILS
AFTER INSERT
AS
BEGIN
    DECLARE @EID INT,@ENAME VARCHAR(250)
    SELECT @EID = EmployeeID FROM inserted
    SELECT @ENAME=EmployeeName FROM inserted

    INSERT INTO EmployeeLogs VALUES(@EID,@ENAME,'INSERT',GETDATE())

END

-----UPDATE-----
CREATE OR ALTER TRIGGER TR_UPDATE_LOG
ON EMPLOYEEDETAILS
AFTER UPDATE
AS
BEGIN
    DECLARE @EID INT,@ENAME VARCHAR(250)
    SELECT @EID = EmployeeID FROM inserted
    SELECT @ENAME=EmployeeName FROM inserted

    INSERT INTO EmployeeLogs VALUES(@EID,@ENAME,'UPDATE',GETDATE())

END

-----DELETE-----
CREATE OR ALTER TRIGGER TR_DELETE_LOG
ON EMPLOYEEDETAILS
AFTER DELETE
AS
BEGIN
    DECLARE @EID INT,@ENAME VARCHAR(250)
    SELECT @EID = EmployeeID FROM deleted
    SELECT @ENAME=EmployeeName FROM deleted

    INSERT INTO EmployeeLogs VALUES(@EID,@ENAME,'DELETE',GETDATE())

END

3. Create a trigger that fires AFTER INSERT to automatically calculate the joining bonus (10% of the salary)
for new employees and update a bonus column in the EmployeeDetails table.
CREATE OR ALTER TRIGGER TR_INSERT_BONUS
ON EMPLOYEEDETAILS
AFTER INSERT
AS
BEGIN
    DECLARE @SAL DECIMAL(8,2),@EID INT
```

```
SELECT @SAL=Salary FROM inserted
SELECT @EID = EmployeeID FROM inserted
UPDATE EMPLOYEEDETAILS
SET Salary=@SAL*1.1
WHERE EmployeeID=@EID
```

END

Part – B

1. Create a trigger to ensure that the JoiningDate is automatically set to the current date if it is NULL.

```
CREATE OR ALTER TRIGGER TR_UPDATE_DATE_EMP
ON EMPLOYEEDETAILS
AFTER INSERT
AS
BEGIN
```

```
    DECLARE @EID INT,@JDATE DATETIME
    SELECT @EID = EmployeeID FROM inserted
    SELECT @JDATE= JoiningDate FROM inserted
```

```
    INSERT
EMPLOYEEDETAILS(EmployeeID,EmployeeName,ContactNo,Department,Salary,JoiningDate)
    SELECT * FROM inserted
    UPDATE EMPLOYEEDETAILS SET JoiningDate=GETDATE() WHERE @JDATE IS NULL
```

INTO

END

Part – C

1. Create a trigger that ensure that ContactNo is valid during insert and update (ContactNo length is 10)

```
CREATE OR ALTER TRIGGER TR_CONTACTNO
ON EMPLOYEEDETAILS
AFTER INSERT,UPDATE
AS
BEGIN
```

```
    DECLARE @EID INT,@CNo VARCHAR(100)
```

```
    SELECT @EID=EmployeeID FROM inserted
    SELECT @CNO=ContactNo FROM inserted
```

```
    IF (LEN(@CNo)!=10)
```

```
        PRINT 'CONTACT NUMBER IS NOT IN 10 DIGIT'
```

```
        UPDATE EMPLOYEEDETAILS SET ContactNo = '1234567890' WHERE
EmployeeID= (SELECT EmployeeID FROM inserted)
    END
```

Instead of Trigger

Movies		
Column_Name	Data Type	Constraints
MovieID	INT	PRIMARY KEY

MovieTitle	VARCHAR(255)	NOT NULL
ReleaseYear	INT	NOT NULL
Genre	VARCHAR(100)	NOT NULL
Rating	DECIMAL(3,1)	NOT NULL
Duration	INT	NOT NULL – (In minutes)

MoviesLog		
Column_Name	Data Type	Constraints
LogID	INT	PRIMARY KEY,AUTO INCREMENT
MovieID	INT	NOT NULL
MovieTitle	VARCHAR(255)	NOT NULL
ActionPerformed	VARCHAR(100)	NOT NULL
ActionDate	DATETIME	NOT NULL

From the above given tables perform the following queries:

Part – A

1. Create an INSTEAD OF trigger that fires on INSERT, UPDATE and DELETE operation on the Movies table. For that, log all operations performed on the Movies table into MoviesLog.

-----INSERT-----

CREATE OR ALTER TRIGGER TR_INSTEAD_MOVEILOG

ON Movies

INSTEAD OF INSERT

AS

BEGIN

DECLARE @MID INT,@MNAME VARCHAR(250)

SELECT @MID = MovieID FROM inserted

SELECT @MNAME=MovieTitle FROM inserted

INSERT INTO MoviesLog VALUES(@MID,@MNAME,'INSERT',GETDATE())

END

-----UPDATE-----

CREATE OR ALTER TRIGGER TR_INSTEAD_UPDATE_MOVEILOG

ON Movies

INSTEAD OF UPDATE

AS

BEGIN

DECLARE @MID INT,@MNAME VARCHAR(250)

SELECT @MID = MovieID FROM inserted

SELECT @MNAME=MovieTitle FROM inserted

INSERT INTO MoviesLog VALUES(@MID,@MNAME,'UPDATE',GETDATE())

END

-----DELETE-----

CREATE OR ALTER TRIGGER TR_INSTEAD_DELETE_MOVEILOG

ON Movies

INSTEAD OF DELETE

AS

BEGIN

```
DECLARE @MID INT,@MNAME VARCHAR(250)
```

```
SELECT @MID = MovieID FROM deleted
```

```
SELECT @MNAME=MovieTitle FROM deleted
```

```
INSERT INTO MoviesLog VALUES(@MID,@MNAME,'DELETE',GETDATE())
```

```
END
```

2. Create a trigger that only allows to insert movies for which Rating is greater than 5.5 .

```
CREATE OR ALTER TRIGGER TR_RATING
```

```
ON Movies
```

```
INSTEAD OF INSERT
```

```
AS
```

```
BEGIN
```

```
DECLARE @MID INT,@RAT DECIMAL(3, 1),@MNAME VARCHAR(255),@RY INT,@GEN  
VARCHAR(100),@DR INT
```

```
SELECT @MID = MovieID,@MNAME= MovieTitle,@RY=ReleaseYear,@GEN=Genre  
,@DR=Duration FROM inserted
```

```
SELECT @RAT= Rating FROM inserted
```

```
IF(@RAT>5.5)
```

```
INSERT INTO Movies VALUES(@MID,@MNAME,@RY,@GEN,@RAT,@DR)
```

```
END
```

3. Create trigger that prevent duplicate 'MovieTitle' of Movies table and log details of it in MoviesLog table.

```
CREATE OR ALTER Trigger tr_prevent_duplicate_MOVIE
```

```
on Movies
```

```
INSTEAD OF INSERT
```

```
AS
```

```
BEGIN
```

```
DECLARE @MID INT ,@MNAME VARCHAR(250)
```

```
SELECT @MID= MovieID FROM inserted
```

```
SELECT @MNAME=MovieTitle FROM inserted
```

```
INSERT INTO Movies(MovieID,MovieTitle,ReleaseYear,Genre,Rating,Duration)
```

```
SELECT * FROM inserted
```

```
WHERE MovieTitle NOT IN(SELECT MovieTitle FROM Movies)
```

```
INSERT INTO MoviesLog VALUES (@MID,@MNAME,'DELETE',GETDATE())
```

```
END
```

Part – B

1. Create trigger that prevents to insert pre-release movies.

```
CREATE OR ALTER TRIGGER TR_PRE_RELEASE
```

```
ON Movies
```

```
INSTEAD OF INSERT
```

```
AS
```

```
BEGIN
```

```
DECLARE @RY INT
```

```
SELECT @RY=ReleaseYear FROM inserted
```

```
IF (@RY>GETDATE())
    PRINT 'Cannot insert pre-release movie. '
```

END

Part – C

1. Develop a trigger to ensure that the Duration of a movie cannot be updated to a value greater than 120 minutes (2 hours) to prevent unrealistic entries.

```
CREATE OR ALTER TRIGGER TR_CHECK_DURATION
```

```
ON MOVIES
```

```
INSTEAD OF UPDATE
```

```
AS
```

```
BEGIN
```

```
    DECLARE @DR INT
```

```
    SELECT @DR=DURATION FROM inserted
```

```
    IF(@DR>120)
```

```
        PRINT 'DURATION CAN NOT BE MORE THAN 120 MINUTES'
```

END

Lab 16

Perform Exception Handling

Customers		
Column_Name	DataType	Constraints
Customer_id	Int	Primary Key
Customer_Name	Varchar (250)	Not Null
Email	Varchar (50)	Unique

Orders		
Column_Name	DataType	Constraints
Order_id	Int	Primary Key
Customer_id	Int	Foreign Key
Order_date	date	Not Null

From the above given tables perform the following queries:

Part – A

1. Handle Divide by Zero Error and Print message like: Error occurs that is - Divide by zero error.

```
BEGIN TRY
```

```
    DECLARE @A INT ,@B INT ,@RESULT INT
```

```
    SET @A=10
```

```
    SET @B=0
```

```
    SET @RESULT=@A/@B
```

```
    PRINT @RESULT
```

```
END TRY
```

```
BEGIN CATCH
```

```
    PRINT 'NOT dIVISIBLE BY ZERO'
```

```
END CATCH
```

2. Try to convert string to integer and handle the error using try...catch block.

```
BEGIN TRY
```

```
    DECLARE @STRING VARCHAR(50) ,@INT INT
```

```
SET @STRING='ABC'
```

```
SET @INT = CAST(@STRING AS INT)
```

```
PRINT @INT
```

```
END TRY
```

```
BEGIN CATCH
```

```
PRINT 'STRING IS NOT CONVERT AS INT'
```

```
END CATCH
```

3. Create a procedure that prints the sum of two numbers: take both numbers as integer & handle exception with all error functions if any one enters string value in numbers otherwise print result.

```
CREATE PROCEDURE PR_ADDITION
```

```
@N1 VARCHAR(50),
```

```
@N2 VARCHAR(50)
```

```
AS
```

```
BEGIN
```

```
    BEGIN TRY
```

```
        DECLARE @NUM1 INT ,@NUM2 INT
```

```
        SET @NUM1=CAST(@N1 AS INT)
```

```
        SET @NUM2=CAST(@N2 AS INT)
```

```
        PRINT (@NUM1+@NUM2)
```

```
    END TRY
```

```
    BEGIN CATCH
```

```
        PRINT ERROR_MESSAGE()
```

```
    END CATCH
```

```
END
```

4. Handle a Primary Key Violation while inserting data into customers table and print the error details such as the error message, error number, severity, and state.

```
CREATE PROCEDURE PR_PR_KEY
```

```
@ID INT,@NAME VARCHAR(50),@EMAIL VARCHAR(50)
```

```
AS
```

```
BEGIN
```

```
    BEGIN TRY
```

```
        INSERT INTO Customers VALUES (@ID,@NAME,@EMAIL)
```

```
    END TRY
```

```
    BEGIN CATCH
```

```
        PRINT ERROR_MESSAGE()
```

```
    END CATCH
```

```
END
```

5. Throw custom exception using stored procedure which accepts Customer_id as input & that throws Error like no Customer_id is available in database.

```
CREATE PROCEDURE PR_AVAILABLE_ID
```

```
@ID INT
```

```
AS
```



```
BEGIN
    IF NOT EXISTS (SELECT Customer_id FROM Customers WHERE Customer_id=@ID)
        THROW 5001,'NO CUSTOMER_ID EXISTS IN DATABASE',1
    ELSE
        PRINT 'CUSTOMER_ID EXISTS '
END
```

Part – B

1. Handle a Foreign Key Violation while inserting data into Orders table and print appropriate error message.

```
BEGIN TRY
    INSERT INTO Orders VALUES (2,102,'2025-05-19')
END TRY

BEGIN CATCH
    PRINT ERROR_MESSAGE()
END CATCH
```

2. Throw custom exception that throws error if the data is invalid.

```
BEGIN
    DECLARE @NUM INT
    SET @NUM=-2
    IF @NUM<0
        THROW 50001,'DATA IS INVALID',1
    ELSE
        PRINT 'NUMBER IS POSITIVE'
END
```

3. Create a Procedure to Update Customer's Email with Error Handling

```
CREATE OR ALTER PROCEDURE PR_UPDATE
@EMAIL VARCHAR(50),@ID INT
AS
BEGIN
    BEGIN TRY
        UPDATE Customers
        SET Email=@EMAIL
        WHERE Customer_id=@ID
    END TRY
    BEGIN CATCH
        PRINT 'CUSTOMER DOES NOT EXISTS'
    END CATCH
END
```

	Part – C 1. Create a procedure which prints the error message that “The Customer_id is already taken. Try another one”. <pre> CREATE OR ALTER PROCEDURE PR_ERR_MSG @C_ID INT,@C_NAME VARCHAR(250),@C_EMAIL VARCHAR(50) AS BEGIN BEGIN TRY INSERT INTO CUSTOMERS VALUES(@C_ID,@C_NAME,@C_EMAIL) END TRY BEGIN CATCH PRINT 'THE CUSTOMER_ID IS ALREADY TAKEN TRY ANOTHER ONE' END CATCH END </pre> 2. Handle Duplicate Email Insertion in Customers Table. <pre> CREATE OR ALTER PROCEDURE PR_ERR_EMAIL_MSG @C_ID INT,@C_NAME VARCHAR(250),@C_EMAIL VARCHAR(50) AS BEGIN BEGIN TRY INSERT INTO CUSTOMERS VALUES(@C_ID,@C_NAME,@C_EMAIL) END TRY BEGIN CATCH PRINT 'THE EMAIL IS ALREADY TAKEN TRY ANOTHER ONE' END CATCH END </pre>
Lab-1 MongoDB	Perform following queries using use, drop, createcollection, dropcollection, insertOne and insertMany method. Part - A 1. Create a new database named “Darshan”. use Darshan 2. Create another new database named “DIET”. use DIET 3. List all databases. show databases

4. Check the current database.
Db
5. Drop “DIET” database.
use DIET
db.dropDatabase()
6. Create a collection named “Student” in the “Darshan” database.
use Darshan
db.createCollection("Student")
7. Create a collection named “Department” in the “Darshan” database.
db.createCollection("Department")
8. List all collections in the “Darshan” database.
show collections
9. Insert a single document using insertOne into “Department” collection. (Dname: 'CE', HOD: 'Patel')
db.Department.insertOne({ Dname: 'CE', HOD: 'Patel' })
10. Insert two document using insertMany into “Department” collection. (Dname: 'IT' and Dname: 'ICT')
db.Department.insertMany([{ Dname: 'IT' }, { Dname: 'ICT' }])
11. Drop a collection named “Department” from the “Darshan” database.
db.Department.drop()
12. Insert a single document using insertOne into “Student” collection.
(Fields are Name, City, Branch, Semester, Age) Insert your own data.
db.Student.insertOne({
 Name: 'Mann',
 City: 'Junagadh',
 Branch: 'CSE',
 Semester: '6',
 Age: 20 })
13. Insert three documents using insertMany into “Student” collection.
(Fields are Name, City, Branch, Semester, Age) Insert your three friend's data.
db.Student.insertMany([
 { Name: 'Friend1', City: 'City1', Branch: 'Branch1', Semester: 'Sem1', Age: 18 },
 { Name: 'Friend2', City: 'City2', Branch: 'Branch2', Semester: 'Sem2', Age: 19 },
 { Name: 'Friend3', City: 'City3', Branch: 'Branch3', Semester: 'Sem3', Age: 20 }
])
14. Check whether “Student” collection exists or not.
db.getCollectionNames().includes("Student")

15. Check the stats of “Student” collection.

```
db.Student.stats()
```

16. Drop the “Student” collection.

```
db.Student.drop()
```

17. Create a collection named “Deposit”.

```
db.createCollection("Deposit")
```

18. Insert following data in to “Deposit” collection.

Deposit				
ACTNO	CNAME	BNAME	AMOUNT	CITY
101	ANIL	VRCE	1000.00	RAJKOT
102	SUNIL	AJNI	5000.00	SURAT
103	MEHUL	KAROLBAGH	3500.00	BARODA
104	MADHURI	CHANDI	1200.00	AHMEDABAD
105	PRMOD	M.G. ROAD	3000.00	SURAT
106	SANDIP	ANDHERI	2000.00	RAJKOT
107	SHIVANI	VIRAR	1000.00	SURAT
108	KRANTI	NEHRU PLACE	5000.00	RAJKOT

```
db.Deposit.insertMany([
```

```
{ ACTNO: 101, CNAME: 'ANIL', BNAME: 'VRCE', AMOUNT: 1000.00, CITY: 'RAJKOT' },
```

```
{ ACTNO: 102, CNAME: 'SUNIL', BNAME: 'AJNI', AMOUNT: 5000.00, CITY: 'SURAT' },
```

```
{ ACTNO: 103, CNAME: 'MEHUL', BNAME: 'KAROLBAGH', AMOUNT: 3500.00, CITY: 'BARODA' },
```

```
{ ACTNO: 104, CNAME: 'MADHURI', BNAME: 'CHANDI', AMOUNT: 1200.00, CITY: 'AHMEDABAD' }, {
```

```
ACTNO: 105, CNAME: 'PRMOD', BNAME: 'M.G. ROAD', AMOUNT: 3000.00, CITY: 'SURAT' },
```

```
{ ACTNO: 106, CNAME: 'SANDIP', BNAME: 'ANDHERI', AMOUNT: 2000.00, CITY: 'RAJKOT' },
```

```
{ ACTNO: 107, CNAME: 'SHIVANI', BNAME: 'VIRAR', AMOUNT: 1000.00, CITY: 'SURAT' },
```

```
{ ACTNO: 108, CNAME: 'KRANTI', BNAME: 'NEHRU PLACE', AMOUNT: 5000.00, CITY: 'RAJKOT' }
```

```
]]
```

19. Display all the documents of “Deposit” collection.

```
db.Deposit.find()
```

20. Drop the “Deposit” collection.

```
db.Deposit.drop()
```

Part – B

1. Create a new database named “Computer”.

```
use Computer
```

2. Create a collection named “Faculty” in the “Computer” database.

```
db.createCollection("Faculty")
```

3. Insert a below document using insertOne into “Faculty” collection.

Faculty				
FID	FNAME	BNAME	SALARY	JDATE

1	ANIL	CE	10000	1-3-95
---	------	----	-------	--------

```
db.Faculty.insertOne({
  FID: 1, FNAME: 'ANIL', BNAME: 'CE', SALARY: 10000, JDATE: '1-3-95'
})
```

4. Insert below documents using insertMany into “Faculty” collection.

Faculty				
FID	FNAME	BNAME	SALARY	JDATE
2	SUNIL	CE	50000	4-1-96
3	MEHUL	IT	35000	17-11-95
4	MADHURI	IT	12000	17-12-95
5	PRMOD	CE	30000	27-3-96
6	SANDIP	CE	20000	31-3-96
7	SHIVANI	CE	10000	5-9-95
8	KRANTI	IT	50000	2-7-95

```
db.Faculty.insertMany([
  { FID: 2, FNAME: 'SUNIL', BNAME: 'CE', SALARY: 50000, JDATE: '4-1-96' },
  { FID: 3, FNAME: 'MEHUL', BNAME: 'IT', SALARY: 35000, JDATE: '17-11-95' },
  { FID: 4, FNAME: 'MADHURI', BNAME: 'IT', SALARY: 12000, JDATE: '17-12-95' },
  { FID: 5, FNAME: 'PRMOD', BNAME: 'CE', SALARY: 30000, JDATE: '27-3-96' },
  { FID: 6, FNAME: 'SANDIP', BNAME: 'CE', SALARY: 20000, JDATE: '31-3-96' },
  { FID: 7, FNAME: 'SHIVANI', BNAME: 'CE', SALARY: 10000, JDATE: '5-9-95' },
  { FID: 8, FNAME: 'KRANTI', BNAME: 'IT', SALARY: 50000, JDATE: '2-7-95' }
])
```

5. Display all the documents of “Faculty” collection.

```
db.Faculty.find()
```

6. Drop the “Faculty” collection.

```
db.Faculty.drop()
```

7. Drop the “Computer” database.

```
use Computer db.dropDatabase()
```

Part – C (Perform following operation using UI)

1. Create a new database named “Computer”

```
db.Deposit.find().
```

2. Create a collection named “Faculty” in the “Computer” database.

```
db.Deposit.findOne()
```

3. Insert a below documents into “Faculty” collection.

Faculty				
FID	FNAME	BNAME	SALARY	JDATE
1	ANIL	CE	10000	1-3-95
2	SUNIL	CE	50000	4-1-96
3	MEHUL	IT	35000	17-11-95

4	MADHURI	IT	12000	17-12-95
5	PRMOD	CE	30000	27-3-96
6	SANDIP	CE	20000	31-3-96
7	SHIVANI	CE	10000	5-9-95
8	KRANTI	IT	50000	2-7-95

```
db.Faculty.insertMany([
  { FID: 2, FNAME: 'SUNIL', BNAME: 'CE', SALARY: 50000, JDATE: '4-1-96' },
  { FID: 3, FNAME: 'MEHUL', BNAME: 'IT', SALARY: 35000, JDATE: '17-11-95' },
  { FID: 4, FNAME: 'MADHURI', BNAME: 'IT', SALARY: 12000, JDATE: '17-12-95' },
  { FID: 5, FNAME: 'PRMOD', BNAME: 'CE', SALARY: 30000, JDATE: '27-3-96' },
  { FID: 6, FNAME: 'SANDIP', BNAME: 'CE', SALARY: 20000, JDATE: '31-3-96' },
  { FID: 7, FNAME: 'SHIVANI', BNAME: 'CE', SALARY: 10000, JDATE: '5-9-95' },
  { FID: 8, FNAME: 'KRANTI', BNAME: 'IT', SALARY: 50000, JDATE: '2-7-95' }
])
```

4. Display all the documents of “Faculty” collection.

```
db.Faculty.find()
```

5. Drop the “Faculty” collection.

```
db.Faculty.drop()
```

6. Drop the “Computer” database.

```
use Computer
```

```
db.dropDatabase()
```

Lab-2
MongoDB

Perform following queries using find, limit, skip and sort method.

Create and Select Database Named: “BANK_INFO”

Deposit (Collection name)				
ACTNO	CNAME	BNAME	AMOUNT	ADATE
101	ANIL	VRCE	1000	1-3-95
102	SUNIL	AJNI	5000	4-1-96
103	MEHUL	KAROLBAGH	3500	17-11-95
104	MADHURI	CHANDI	1200	17-12-95
105	PRMOD	M.G. ROAD	3000	27-3-96
106	SANDIP	ANDHERI	2000	31-3-96
107	SHIVANI	VIRAR	1000	5-9-95
108	KRANTI	NEHRU PLACE	5000	2-7-95

From the above given collection perform the following queries using find, limit, skip and sort method:

Part - A

1. Retrieve/Display every document of Deposit collection.

```
db.Deposit.find()
```

2. Display only one document of Deposit collection. (Use: findOne())

```
db.Deposit.findOne()
```

3. Insert following document into Deposit collection. (Use: insertOne())

109	KIRTI	VIRAR	3000	3-5-97
-----	-------	-------	------	--------

```
db.Deposit.insertOne({ ACTNO: 109, CNAME: 'KIRTI', BNAME: 'VIRAR', AMOUNT: 3000, ADATE: '3-5-97' })
```

4. Insert following documents into Deposit collection. (Use: insertMany())

110	MITALI	ANDHERI	4500	4-9-95
111	RAJIV	NEHRU PLACE	7000	2-10-98

db.Deposit.insertMany([

{ ACTNO: 110, CNAME: 'MITALI', BNAME: 'ANDHERI', AMOUNT: 4500, ADATE: '4-9-95' },
{ ACTNO: 111, CNAME: 'RAJIV', BNAME: 'NEHRU PLACE', AMOUNT: 7000, ADATE: '2-10-98' }])

5. Display all the documents of 'VIRAR' branch from Deposit collection.

db.Deposit.find({ BNAME: 'VIRAR' })

6. Display all the documents of Deposit collection whose amount is between 3000 and 5000.

db.Deposit.find({ AMOUNT: { \$gte: 3000, \$lte: 5000 } })

7. Display all the documents of Deposit collection whose amount is greater than 2000 and branch is VIRAR.

db.Deposit.find({ AMOUNT: { \$gt: 2000 }, BNAME: 'VIRAR' })

8. Display all the documents with CNAME, BNAME and AMOUNT fields from Deposit collection.

db.Deposit.find({}, { CNAME: 1, BNAME: 1, AMOUNT: 1, _id: 0 })

9. Display all the documents of Deposit collection on ascending order by CNAME.

db.Deposit.find().sort({ CNAME: 1 })

10. Display all the documents of Deposit collection on descending order by BNAME.

db.Deposit.find().sort({ BNAME: -1 })

11. Display all the documents of Deposit collection on ascending order by ACTNO and descending order by AMOUNT.

db.Deposit.find().sort({ ACTNO: 1, AMOUNT: -1 })

12. Display only two documents of Deposit collection.

db.Deposit.find().limit(2)

13. Display 3rd document of Deposit collection.

db.Deposit.find().skip(2).limit(1)

14. Display 6th and 7th documents of Deposit collection.

db.Deposit.find().skip(5).limit(2)

15. Display the count of documents in Deposit collection.

db.Deposit.countDocuments()

Part- B

1. Insert following documents into "Student" collection. (Use: insertMany())

```
{ "_id": 1, "name": "John", "age": 30, "city": "New York", "isActive": true }
{ "_id": 2, "name": "Jane", "age": 25, "city": "Los Angeles", "isActive": false }
{ "_id": 3, "name": "Tom", "age": 35, "city": "Chicago", "isActive": true }
{ "_id": 4, "name": "Lucy", "age": 28, "city": "San Francisco", "isActive": true }
```

```
{ "_id": 5, "name": "David", "age": 40, "city": "Miami", "isActive": false }
{ "_id": 6, "name": "Eva", "age": 23, "city": "Boston", "isActive": true }
{ "_id": 7, "name": "Nick", "age": 38, "city": "Seattle", "isActive": false }
{ "_id": 8, "name": "Sophia", "age": 27, "city": "New York", "isActive": true }
{ "_id": 9, "name": "Liam", "age": 32, "city": "Los Angeles", "isActive": false }
{ "_id": 10, "name": "Olivia", "age": 29, "city": "San Diego", "isActive": true }
```

```
db.Student.insertMany([ { _id: 1, name: "John", age: 30, city: "New York", isActive: true }, { _id: 2, name:
"Jane", age: 25, city: "Los Angeles", isActive: false }, { _id: 3, name: "Tom", age: 35, city: "Chicago",
isActive: true }, { _id: 4, name: "Lucy", age: 28, city: "San Francisco", isActive: true }, { _id: 5, name:
"David", age: 40, city: "Miami", isActive: false }, { _id: 6, name: "Eva", age: 23, city: "Boston", isActive:
true }, { _id: 7, name: "Nick", age: 38, city: "Seattle", isActive: false }, { _id: 8, name: "Sophia", age: 27,
city: "New York", isActive: true }, { _id: 9, name: "Liam", age: 32, city: "Los Angeles", isActive: false }, {
_id: 10, name: "Olivia", age: 29, city: "San Diego", isActive: true } ])
```

2. Display all documents of "Student" collection.

```
db.Student.find()
```

3. Display all documents of "Student" collection whose age is 30.

```
db.Student.find({ age: 30 })
```

4. Display all documents of "Student" collection whose age is greater than 25.

```
db.Student.find({ age: { $gt: 25 } })
```

5. Display all documents of "Student" collection whose name is "John" and age is 30.

```
db.Student.find({ name: "John", age: 30 })
```

6. Display all documents of "Student" collection whose age is not equal to 25.

```
db.Student.find({ age: { $ne: 25 } })
```

7. Display all documents of "Student" collection whose age is equal to 25 or 30 or 35. (using \$or as well as using \$in).

```
db.Student.find({ $or: [{ age: 25 }, { age: 30 }, { age: 35 } ] })
```

```
db.Student.find({ age: { $in: [25, 30, 35] } })
```

8. Display all documents of "Student" collection whose name is "John" or age is 30.

```
db.Student.find({ $or: [{ name: "John" }, { age: 30 } ] })
```

9. Display all documents of "Student" collection whose name is "John" and city is New York.

```
db.Student.find({ name: "John", city: "New York" })
```

10. Display name and age of students from "Student" collection whose name is "John" and city is New York.

```
db.Student.find({ name: "John", city: "New York" }, { name: 1, age: 1, _id: 0 })
```

Part – C

1. Display name of students from "Student" collection whose age is between to 25 and 35 and sort output by age in ascending order.

```
db.Student.find({ age: { $gte: 25, $lte: 35 } }, { name: 1, _id: 0 }).sort({ age: 1 })
```


	- Display all documents of "Student" collection and sort all the documents by name in ascending order and then by age in descending. <code>db.Student.find().sort({ name: 1, age: -1 })</code> - Display first five documents of "Student" collection. <code>db.Student.find().limit(5)</code> - Display fourth and fifth documents of "Student" collection. <code>db.Student.find().skip(3).limit(2)</code> - Display the name of oldest student from "Student" collection. <code>db.Student.find().sort({ age: -1 }).limit(1).project({ name: 1, _id: 0 })</code> - Display all documents of "Student" collection in such a way that skip the first 2 documents and return the rest documents. <code>db.Student.find().skip(2)</code>
Lab-3 MongoDB	Perform the following queries using update, delete, rename and createcollection method: Part – A (Use collection "Student" created in Lab-2) - Update the age of John's to 31. <code>db.Student.updateOne({ name: "John" }, { \$set: { age: 31 } })</code> - Update the city of all students from 'New York' to 'New Jersey'. <code>db.Student.updateMany({ city: "New York" }, { \$set: { city: "New Jersey" } })</code> - Set isActive to false for every student older than 35. <code>db.Student.updateMany({ age: { \$gt: 35 } }, { \$set: { isActive: false } })</code> - Increment the age of all students by 1 year. <code>db.Student.updateMany({}, { \$inc: { age: 1 } })</code> - Set the city of 'Eva' to 'Cambridge'. <code>db.Student.updateOne({ name: "Eva" }, { \$set: { city: "Cambridge" } })</code> - Update 'Sophia's isActive status to false. <code>db.Student.updateOne({ name: "Sophia" }, { \$set: { isActive: false } })</code> - Update the city field of student aged below 30 to 'San Diego'. <code>db.Student.updateMany({ age: { \$lt: 30 } }, { \$set: { city: "San Diego" } })</code> - Rename the age field to years for all documents. <code>db.Student.updateMany({}, { \$rename: { "age": "years" } })</code> - Update 'Nick' to make him active (isActive = true). <code>db.Student.updateOne({ name: "Nick" }, { \$set: { isActive: true } })</code> - Update all documents to add a new field country with the value 'USA'. <code>db.Student.updateMany({}, { \$set: { country: "USA" } })</code>

11. Update 'David's city to 'Orlando'.
`db.Student.updateOne({ name: "David" }, { $set: { city: "Orlando" } })`
12. Multiply the age of all students by 2.
`db.Student.updateMany({}, { $mul: { years: 2 } })`
13. Unset (remove) the city field for 'Tom'.
`db.Student.updateOne({ name: "Tom" }, { $unset: { city: "" } })`
14. Add a new field premiumUser and to true for users older than 30.
`db.Student.updateMany({ years: { $gt: 30 } }, { $set: { premiumUser: true } })`
15. Set isActive to true for 'Jane'.
`db.Student.updateOne({ name: "Jane" }, { $set: { isActive: true } })`
16. Update isActive field of 'Lucy' to false.
`db.Student.updateOne({ name: "Lucy" }, { $set: { isActive: false } })`
17. Delete a document of 'Nick' from the collection.
`db.Student.deleteOne({ name: "Nick" })`
18. Delete all students who are inactive (isActive = false).
`db.Student.deleteMany({ isActive: false })`
19. Delete all students who live in 'New York'.
`db.Student.deleteMany({ city: "New York" })`
20. Delete all the students aged above 35.
`db.Student.deleteMany({ years: { $gt: 35 } })`
21. Delete a student named "Olivia" from the collection.
`db.Student.deleteOne({ name: "Olivia" })`
22. Delete all the students whose age is below 25.
`db.Student.deleteMany({ years: { $lt: 25 } })`
23. Delete the first student whose isActive field is true.
`db.Student.deleteOne({ isActive: true })`
24. Delete all students from 'Los Angeles'.
`db.Student.deleteMany({ city: "Los Angeles" })`
25. Delete all students who have city field missing.
`db.Student.deleteMany({ city: { $exists: false } })`
26. Rename 'city' field to 'location' for all documents.
`db.Student.updateMany({}, { $rename: { "city": "location" } })`

27. Rename the name field to FullName for 'John'.
`db.Student.updateOne({ name: "John" }, { $rename: { "name": "FullName" } })`
28. Rename the isActive field to status for all documents.
`db.Student.updateMany({}, { $rename: { "isActive": "status" } })`
29. Rename age to yearsOld for everyone from 'San Francisco' student only.
`db.Student.updateMany({ location: "San Francisco" }, { $rename: { "years": "yearsOld" } })`
30. Create a Capped Collection named “Employee” as per follows:
a. Ecode and Ename are compulsory fields
b. Datatype of EID is int, Ename is string, Age is int and City is string
Insert following documents into above “Employee” collection.
`{"Ecode": 1, "Ename": "John"}`
`{"Ecode": 2, "Ename": "Jane", "age": 25, "city": "Los Angeles"}`
`{"Ecode": 3, "Ename": "Tom", "age": 35}`
`{"Ecode": 4, "Ename": "Lucy", "age": 28, "city": "San Francisco", "isActive": true}`
`{"Ename": "Dino"}`
- ```
db.createCollection("Employee", {
 capped: true,
 size: 5120,
 max: 100,
 validator: {
 $jsonSchema: {
 bsonType: "object",
 required: ["Ecode", "Ename"],
 properties: {
 }
 }
 })
db.Employee.insertMany([
 { Ecode: 1, Ename: "John" },
 Ecode: { bsonType: "int" },
 Ename: { bsonType: "string" },
 Age: { bsonType: "int" },
 City: { bsonType: "string" }
 { Ecode: 2, Ename: "Jane", age: 25, city: "Los Angeles" },
 { Ecode: 3, Ename: "Tom", age: 35 },
 { Ecode: 4, Ename: "Lucy", age: 28, city: "San Francisco", isActive: true },
 { Ename: "Dino" }
])
```

**Part – B Create collection named “Student\_data” and insert following 10 documents into it.**

| Student_data |         |            |       |     |        |           |
|--------------|---------|------------|-------|-----|--------|-----------|
| ROLLNO       | SNAME   | DEPARTMENT | FEES  | SEM | GENDER | CITY      |
| 101          | Vina    | CE         | 15000 | 3   | Female | Rajkot    |
| 102          | Krishna | EC         | 8000  | 5   | Female | Ahmedabad |
| 103          | Priti   | Civil      | 12000 | 7   | Female | Baroda    |
| 104          | Mitul   | CE         | 15000 | 3   | Male   | Rajkot    |
| 105          | Keshav  | CE         | 15000 | 3   | Male   | Jamnagar  |
| 106          | Zarna   | Civil      | 12000 | 5   | Female | Ahmedabad |
| 107          | Nima    | EE         | 9000  | 5   | Female | Rajkot    |
| 108          | Dhruv   | Mechanical | 10000 | 5   | Male   | Rajkot    |
| 109          | Krish   | Mechanical | 10000 | 7   | Male   | Baroda    |
| 110          | Zeel    | EE         | 9000  | 3   | Female | Jamnagar  |

**From the above given "Student\_data" collection perform the following queries:**

- Display Female students and belong to Rajkot city.  
db.Student\_data.find({ GENDER: "Female", CITY: "Rajkot" })
- Display students not studying in 3<sup>rd</sup> sem.  
db.Student\_data.find({ SEM: { \$ne: 3 } })
- Display students whose city is Jamnagar or Baroda. (use: IN)  
db.Student\_data.find({ CITY: { \$in: ["Jamnagar", "Baroda"] } })
- Display first 2 students names who lives in Baroda.  
db.Student\_data.find({ CITY: "Baroda" }).limit(2).project({ SNAME: 1, \_id: 0 })
- Display Male students who studying in 3<sup>rd</sup> sem.  
db.Student\_data.find({ GENDER: "Male", SEM: 3 })
- Display sname and city and fees of those students whose roll no is less than 105.  
db.Student\_data.find({ ROLLNO: { \$lt: 105 } }, { SNAME: 1, CITY: 1, FEES: 1, \_id: 0 })
- Update City of all students from 'Jamnagar' City and Department as 'CE' to 'Surat'.  
db.Student\_data.updateMany({ CITY: "Jamnagar", DEPARTMENT: "CE" }, { \$set: { CITY: "Surat" } })
- Increase Fees by 500 where the Gender is not 'Female'. (Use: Not)  
db.Student\_data.updateMany({ GENDER: { \$ne: "Female" } }, { \$inc: { FEES: 500 } })
- Set the Department of all students from 'EE' and in Sem 3 to 'Electrical'.  
db.Student\_data.updateMany({ DEPARTMENT: "EE", SEM: 3 }, { \$set: { DEPARTMENT: "Electrical" } })
- Update the Fees of students in 'Rajkot' who are male.  
db.Student\_data.updateMany({ CITY: "Rajkot", GENDER: "Male" }, { \$set: { FEES: 11000 } })
- Change City to 'Vadodara' for students in Sem 5 and with fees less than 10000.  
db.Student\_data.updateMany({ SEM: 5, FEES: { \$lt: 10000 } }, { \$set: { CITY: "Vadodara" } })
- Delete all students where the City is 'Ahmedabad' or GENDER is 'Male'.

```
db.Student_data.deleteMany({ $or: [{ CITY: "Ahmedabad" }, { GENDER: "Male" }] })
```

13. Delete students whose Rollno is not in the list [101, 105, 110].

```
db.Student_data.deleteMany({ ROLLNO: { $nin: [101, 105, 110] } })
```

14. Delete students from the 'Civil' department who are in Sem 5 or Sem 7.

```
db.Student_data.deleteMany({ DEPARTMENT: "Civil", SEM: { $in: [5, 7] } })
```

15. Delete all students who are not in the cities 'Rajkot', 'Baroda', or 'Jamnagar'.

```
db.Student_data.deleteMany({ CITY: { $nin: ["Rajkot", "Baroda", "Jamnagar"] } })
```

16. Delete students whose Rollno is between 105 and 108.

```
db.Student_data.deleteMany({ ROLLNO: { $gte: 105, $lte: 108 } })
```

17. Rename the City field to LOCATION for all students.

```
db.Student_data.updateMany({}, { $rename: { "CITY": "LOCATION" } })
```

18. Rename the Department field to Branch where the Fees is less than 10000.

```
db.Student_data.updateMany({ FEES: { $lt: 10000 } }, { $rename: { "DEPARTMENT": "Branch" } })
```

19. Rename Sname to Fullname for students with Rollno in [106, 107, 108].

```
db.Student_data.updateMany({ ROLLNO: { $in: [106, 107, 108] } }, { $rename: { "SNAME": "Fullname" } })
```

20. Rename Fees to Tuition\_Fees for all students with Fees greater than 9000.

```
db.Student_data.updateMany({ FEES: { $gt: 9000 } }, { $rename: { "FEES": "Tuition_Fees" } })
```

21. Rename Department to Major where the Fees is less than 15000 and Gender is 'Female'.

```
db.Student_data.updateMany({ FEES: { $lt: 15000 }, GENDER: "Female" }, { $rename: { "DEPARTMENT": "Major" } })
```

22. Rename City to Hometown for all students whose SEM is 3 and Department is not 'Mechanical'.

```
db.Student_data.updateMany({ SEM: 3, DEPARTMENT: { $ne: "Mechanical" } }, { $rename: { "CITY": "Hometown" } })
```

### Part – C

1. Create a capped collection named "logs" with a maximum size of 100 KB and a maximum of 10 documents.

```
db.createCollection("logs", { capped: true, size: 102400, max: 10 })
```

2. Insert below 12 log entries into the "logs" collection. Each entry should contain a message, level (e.g., "info", "warning", "error"), and a timestamp field. Use the insertMany() method.

```
{ message: "System started", level: "info", timestamp: new Date() }
```

```
{ message: "Disk space low", level: "warning", timestamp: new Date() }
```

```
{ message: "User login", level: "info", timestamp: new Date() }
```

```
{ message: "System reboot", level: "info", timestamp: new Date() }
```

```
{ message: "Error in module", level: "error", timestamp: new Date() }
```

```
{ message: "Memory usage high", level: "warning", timestamp: new Date() }
```

```
{ message: "User logout", level: "info", timestamp: new Date() }
```

```
{ message: "File uploaded", level: "info", timestamp: new Date() }
```

```
{ message: "Network error", level: "error", timestamp: new Date() }
{ message: "Backup completed", level: "info", timestamp: new Date() }
{ message: "Database error", level: "error", timestamp: new Date() }
{ message: "Service started", level: "info", timestamp: new Date() }

db.logs.insertMany([{ message: "System started", level: "info", timestamp: new Date() },
{ message: "Disk space low", level: "warning", timestamp: new Date() },
{ message: "User login", level: "info", timestamp: new Date() },
{ message: "System reboot", level: "info", timestamp: new Date() },
{ message: "Error in module", level: "error", timestamp: new Date() },
{ message: "Memory usage high", level: "warning", timestamp: new Date() },
{ message: "User logout", level: "info", timestamp: new Date() },
{ message: "File uploaded", level: "info", timestamp: new Date() },
{ message: "Network error", level: "error", timestamp: new Date() },
{ message: "Backup completed", level: "info", timestamp: new Date() },
{ message: "Database error", level: "error", timestamp: new Date() },
{ message: "Service started", level: "info", timestamp: new Date() }
])
```

3. Perform find method on “logs” collection to ensure only the **last 10 documents** are retained (even though you inserted 12).

```
db.logs.find()
```

4. Insert below 5 more documents and check if the oldest ones are automatically removed.

```
{ message: "New log entry 1", level: "info", timestamp: new Date() }
{ message: "New log entry 2", level: "info", timestamp: new Date() }
{ message: "New log entry 3", level: "info", timestamp: new Date() }
{ message: "New log entry 4", level: "warning", timestamp: new Date() }
{ message: "New log entry 5", level: "error", timestamp: new Date() }
```

```
db.logs.insertMany([
{ message: "New log entry 1", level: "info", timestamp: new Date() },
{ message: "New log entry 2", level: "info", timestamp: new Date() },
{ message: "New log entry 3", level: "info", timestamp: new Date() },
{ message: "New log entry 4", level: "warning", timestamp: new Date() },
{ message: "New log entry 5", level: "error", timestamp: new Date() }])
```

**Lab-4**  
**MongoDB**

**Perform the following queries using Regex:**

**Part – A Create collection named “Employee” and insert following 10 documents into it.**

| employee |         |        |             |        |          |
|----------|---------|--------|-------------|--------|----------|
| EID      | ENAME   | GENDER | JOININGDATE | SALARY | CITY     |
| 1        | Nick    | Male   | 01-JAN-13   | 4000   | London   |
| 2        | Julian  | Female | 01-OCT-14   | 3000   | New York |
| 3        | Roy     | Male   | 01-JUN-16   | 3500   | London   |
| 4        | Tom     | Male   | NULL        | 4500   | London   |
| 5        | Jerry   | Male   | 01-FEB-13   | 2800   | Sydney   |
| 6        | Philip  | Male   | 01-JAN-15   | 7000   | New York |
| 7        | Sara    | Female | 01-AUG-17   | 4800   | Sydney   |
| 8        | Emily   | Female | 01-JAN-15   | 5500   | New York |
| 9        | Michael | Male   | NULL        | 6500   | London   |

|    |      |      |           |      |        |
|----|------|------|-----------|------|--------|
| 10 | John | Male | 01-JAN-15 | 8800 | London |
|----|------|------|-----------|------|--------|

- Find employees whose name start with E.  
`db.Employee.find({ ENAME: /^E/ })`
- Find employees whose name ends with n.  
`db.Employee.find({ ENAME: /n$/ })`
- Find employees whose name starts with S or M in your collection.  
`db.Employee.find({ ENAME: /^[SM]/ })`
- Find employees where city starts with A to M in your collection.  
`db.Employee.find({ CITY: /^[A-M]/ })`
- Find employees where city name ends in 'ney'.  
`db.Employee.find({ CITY: /ney$/ })`
- Display employee info whose name contains n. (Both uppercase(N) and lowercase(n))  
`db.Employee.find({ ENAME: /n/i })`
- Display employee info whose name starts with E and having 5 characters.  
`db.Employee.find({ ENAME: /^E.{4}$/ })`
- Display employee whose name start with S and ends in a.  
`db.Employee.find({ ENAME: /^S.*a$/ })`
- Display EID, ENAME, CITY and SALARY whose name starts with 'Phi'.  
`db.Employee.find({ ENAME: /^Phi/ }, { EID: 1, ENAME: 1, CITY: 1, SALARY: 1 })`
- Display ENAME, JOININGDATE and CITY whose city contains 'dne' as three letters in city name.  
`db.Employee.find({ CITY: /dne/ }, { ENAME: 1, JOININGDATE: 1, CITY: 1 })`
- Display ENAME, JOININGDATE and CITY who does not belongs to city London or Sydney.  
`db.Employee.find({ CITY: { $nin: ["London", "Sydney"] } }, { ENAME: 1, JOININGDATE: 1, CITY: 1 })`
- Find employees whose names start with 'J'.  
`db.Employee.find({ ENAME: /^J/ })`
- Find employees whose names end with 'y'.  
`db.Employee.find({ ENAME: /y$/ })`
- Find employees whose names contain the letter 'a'.  
`db.Employee.find({ ENAME: /a/ })`
- Find employees whose names contain either 'a' or 'e'.  
`db.Employee.find({ ENAME: /[ae]/ })`

16. Find employees whose names start with 'J' and end with 'n'.  
db.Employee.find({ ENAME: /^J.\*n\$/ })
17. Find employees whose CITY starts with 'New'.  
db.Employee.find({ CITY: /^New/ })
18. Find employees whose CITY does not start with 'L'.  
db.Employee.find({ CITY: { \$not: /^L/ } })
19. Find employees whose CITY contains the word 'York'.  
db.Employee.find({ CITY: /York/ })
20. Find employees whose names have two consecutive vowels (a, e, i, o, u).  
db.Employee.find({ ENAME: /[aeiou]{2}/ })
21. Find employees whose names have three or more letters.  
db.Employee.find({ ENAME: /^.{3,}\$/ })
22. Find employees whose names have exactly 4 letters.  
db.Employee.find({ ENAME: /^.{4}\$/ })
23. Find employees whose names start with either 'S' or 'M'.  
db.Employee.find({ ENAME: /^[SM]/ })
24. Find employees whose names contain 'il' anywhere.  
db.Employee.find({ ENAME: /il/ })
25. Find employees whose names do not contain 'a'.  
db.Employee.find({ ENAME: { \$not: /a/ } })
26. Find employees whose names contain any digit.  
db.Employee.find({ ENAME: /\d/ })
27. Find employees whose names contain exactly one vowel.  
db.Employee.find({ ENAME: /^[^aeiou]\*[aeiou][^aeiou]\*\$/i })
28. Find employees whose names start with any uppercase letter followed by any lowercase letter.  
db.Employee.find({ ENAME: /^[A-Z][a-z]/ })

**Part – B Create collection named “Student” and insert following 10 documents into it.**

| Student |         |            |       |     |        |           |
|---------|---------|------------|-------|-----|--------|-----------|
| ROLLNO  | SNAME   | DEPARTMENT | FEES  | SEM | GENDER | CITY      |
| 101     | Vina    | CE         | 15000 | 3   | Female | Rajkot    |
| 102     | Krishna | EC         | 8000  | 5   | Female | Ahmedabad |
| 103     | Priti   | Civil      | 12000 | 7   | Female | Baroda    |
| 104     | Mitul   | CE         | 15000 | 3   | Male   | Rajkot    |
| 105     | Keshav  | CE         | 15000 | 3   | Male   | Jamnagar  |
| 106     | Zarna   | Civil      | 12000 | 5   | Female | Ahmedabad |



|     |       |            |       |   |        |          |
|-----|-------|------------|-------|---|--------|----------|
| 107 | Nima  | EE         | 9000  | 5 | Female | Rajkot   |
| 108 | Dhruv | Mechanical | 10000 | 5 | Male   | Rajkot   |
| 109 | Krish | Mechanical | 10000 | 7 | Male   | Baroda   |
| 110 | Zeel  | EE         | 9000  | 3 | Female | Jamnagar |

1. Display documents where sname start with K.  
db.Student.find({ SNAME: /^K/ })
2. Display documents where sname starts with Z or D.  
db.Student.find({ SNAME: /^[ZD]/ })
3. Display documents where city starts with A to R.  
db.Student.find({ CITY: /^[A-R]/ })
4. Display students' info whose name start with P and ends with i.  
db.Student.find({ SNAME: /^P.\*i\$/ })
5. Display students' info whose department name starts with 'C'.  
db.Student.find({ DEPARTMENT: /^C/ })
6. Display name, sem, fees, and department whose city contains 'med' as three letters somewhere in city name.  
db.Student.find({ CITY: /med/ }, { SNAME: 1, SEM: 1, FEES: 1, DEPARTMENT: 1 })
7. Display name, sem, fees, and department who does not belongs to city Rajkot or Baroda.  
db.Student.find({ CITY: { \$nin: ["Rajkot", "Baroda"] } }, { SNAME: 1, SEM: 1, FEES: 1, DEPARTMENT: 1 })
8. Find students whose names start with 'K' and are followed by any character.  
db.Student.find({ SNAME: /^K./ })
9. Find students whose names end with 'a'.  
db.Student.find({ SNAME: /a\$/ })
10. Find students whose names contain 'ri'. (case-insensitive)  
db.Student.find({ SNAME: /ri/i })

#### Part – C

1. Find students whose names start with a vowel (A, E, I, O, U).  
db.Student.find({ SNAME: /^[AEIOU]/ })
2. Find students whose CITY ends with 'pur' or 'bad'.  
db.Student.find({ CITY: /(pur|bad)\$/ })
3. Find students whose FEES starts with '1'.  
db.Student.find({ FEES: /^1/ })
4. Find students whose SNAME starts with 'K' or 'V'.  
db.Student.find({ SNAME: /^[KV]/ })

5. Find students whose CITY contains exactly five characters.  
db.Student.find({ CITY: /^. {5}\$/ })
6. Find students whose names do not contain the letter 'e'.  
db.Student.find({ SNAME: { \$not: /e/ } })
7. Find students whose CITY starts with 'Ra' and ends with 'ot'.  
db.Student.find({ CITY: /^Ra.\*ot\$/ })
8. Find students whose names contain exactly one vowel.  
db.Student.find({ SNAME: /^[^aeiou]\*[aeiou][^aeiou]\*\$/i })
9. Find students whose names start and end with the same letter.  
db.Student.find({ SNAME: /^(.).\*\1\$/ })
10. Find students whose DEPARTMENT starts with either 'C' or 'E'.  
db.Student.find({ DEPARTMENT: /^[CE]/ })
11. Find students whose SNAME has exactly 5 characters.  
db.Student.find({ SNAME: /^. {5}\$/ })
12. Find students whose GENDER is Female and CITY starts with 'A'.  
db.Student.find({ GENDER: "Female", CITY: /^A/ })

**Lab-5**  
**MongoDB**

**Perform the following queries using Aggregate:**  
**Part – A Create collection named “Student” and insert following 10 documents into it.**

| Student |         |            |       |     |        |           |
|---------|---------|------------|-------|-----|--------|-----------|
| ROLLNO  | SNAME   | DEPARTMENT | FEES  | SEM | GENDER | CITY      |
| 101     | Vina    | CE         | 15000 | 3   | Female | Rajkot    |
| 102     | Krishna | EC         | 8000  | 5   | Female | Ahmedabad |
| 103     | Priti   | Civil      | 12000 | 7   | Female | Baroda    |
| 104     | Mitul   | CE         | 15000 | 3   | Male   | Rajkot    |
| 105     | Keshav  | CE         | 15000 | 3   | Male   | Jamnagar  |
| 106     | Zarna   | Civil      | 12000 | 5   | Female | Ahmedabad |
| 107     | Nima    | EE         | 9000  | 5   | Female | Rajkot    |
| 108     | Dhruv   | Mechanical | 10000 | 5   | Male   | Rajkot    |
| 109     | Krish   | Mechanical | 10000 | 7   | Male   | Baroda    |
| 110     | Zeel    | EE         | 9000  | 3   | Female | Jamnagar  |

11. Display distinct city.  
db.Student.aggregate([ { \$group: { \_id: "\$CITY" } }, { \$project: { \_id: 0, city: "\$\_id" } } ])
12. Display city wise count of number of students.  
db.Student.aggregate([ { \$group: { \_id: "\$CITY", count: { \$sum: 1 } } } ])
13. Display sum of salary in your collection.  
db.Student.aggregate([ { \$group: { \_id: null, totalSalary: { \$sum: "\$SALARY" } } } ])

14. Display average of salary in your document.  
db.Student.aggregate([{\$group: { \_id: null, avgSalary: { \$avg: "\$SALARY" } } }])
15. Display maximum and minimum salary of your document.  
db.Student.aggregate([{\$group: { \_id: null, maxSalary: { \$max: "\$SALARY" }, minSalary: { \$min: "\$SALARY" } } }])
16. Display city wise total salary in your collection.  
db.Student.aggregate([{\$group: { \_id: "\$CITY", totalSalary: { \$sum: "\$SALARY" } } }])
17. Display gender wise maximum salary in your collection.  
db.Student.aggregate([{\$group: { \_id: "\$GENDER", maxSalary: { \$max: "\$SALARY" } } }])
18. Display city wise maximum and minimum salary.  
db.Student.aggregate([{\$group: { \_id: "\$CITY", maxSalary: { \$max: "\$SALARY" }, minSalary: { \$min: "\$SALARY" } } }])
19. Display count of persons lives in Sydney city in your collection.  
db.Student.aggregate([{\$match: { CITY: "Sydney" } }, { \$count: "count" } ])
20. Display average salary of New York city.  
db.Student.aggregate([{\$match: { CITY: "New York" } }, { \$group: { \_id: "\$CITY", avgSalary: { \$avg: "\$SALARY" } } }])
21. Count the number of male and female students in each Department  
db.Student.aggregate([{\$group: { \_id: { Department: "\$DEPARTMENT", Gender: "\$GENDER" }, count: { \$sum: 1 } } }])
22. Find the total Fees collected from each Department.  
db.Student.aggregate([{\$group: { \_id: "\$DEPARTMENT", totalFees: { \$sum: "\$FEES" } } }])
23. Find the minimum Fees paid by male and female students in each City.  
db.Student.aggregate([{\$group: { \_id: { City: "\$CITY", Gender: "\$GENDER" }, minFees: { \$min: "\$FEES" } } }])
24. Sort students by Fees in descending order and return the top 5.  
db.Student.find().sort({ FEES: -1 }).limit(5)
25. Group students by City and calculate the average Fees for each city, only including cities with more than 1 student.  
db.Student.aggregate([{\$group: { \_id: "\$CITY", avgFees: { \$avg: "\$FEES" }, count: { \$sum: 1 } } }, { \$match: { count: { \$gt: 1 } } }])
26. Filter students from CE or Mechanical department, then calculate the total Fees.  
db.Student.aggregate([{\$match: { DEPARTMENT: { \$in: ["CE", "Mechanical"] } } }, { \$group: { \_id: null, totalFees: { \$sum: "\$FEES" } } }])

27. Count the number of male and female students in each Department.

```
db.Student.aggregate([{$group: { _id: { Department: "$DEPARTMENT", Gender: "$GENDER" }, count: { $sum: 1 } } }])
```

28. Filter students from Rajkot, then group by Department and find the average Fees for each department.

```
db.Student.aggregate([{$match: { CITY: "Rajkot" } }, {$group: { _id: "$DEPARTMENT", avgFees: { $avg: "$FEES" } } }])
```

29. Group by Sem and calculate both the total and average Fees, then sort by total fees in descending order.

```
db.Student.aggregate([{$group: { _id: "$SEM", totalFees: { $sum: "$FEES" }, avgFees: { $avg: "$FEES" } } }, {$sort: { totalFees: -1 } }])
```

30. Find the top 3 cities with the highest total Fees collected by summing up all students' fees in those cities.

```
db.Student.aggregate([{$group: { _id: "$CITY", totalFees: { $sum: "$FEES" } } }, {$sort: { totalFees: -1 } }, {$limit: 3 }])
```

#### **Part – B**

1. Create a collection named "Stock."

```
db.createCollection("Stock")
```

2. Insert below 9 documents into the "Stock" collection.

```
{ "_id": 1,
 "company": "Company-A",
 "sector": "Technology",
 "eps": 5.2,
 "pe": 15.3,
 "roe": 12.8,
 "sales": 300000,
 "profit": 25000
}
{ "_id": 2,
 "company": "Company-B",
 "sector": "Finance",
 "eps": 7.1,
 "pe": 12.4,
 "roe": 10.9,
 "sales": 500000,
 "profit": 55000
}
{ "_id": 3,
 "company": "Company-C",
 "sector": "Retail",
 "eps": 3.8,
 "pe": 22.1,
 "roe": 9.5,
 "sales": 200000,
 "profit": 15000
}
```



```
}
{ "_id": 4,
 "company": "Company-D",
 "sector": "Technology",
 "eps": 5.2,
 "pe": 15.3,
 "roe": 12.8,
 "sales": 300000,
 "profit": 25000
}
{ "_id": 5,
 "company": "Company-E",
 "sector": "Finance",
 "eps": 7.1,
 "pe": 12.4,
 "roe": 10.9,
 "sales": 450000,
 "profit": 40000
}
{ "_id": 6,
 "company": "Company-F",
 "sector": "Healthcare",
 "eps": 3.8,
 "pe": 18.9,
 "roe": 9.5,
 "sales": 500000,
 "profit": 35000
}
{ "_id": 7,
 "company": "Company-G",
 "sector": "Retail",
 "eps": 4.3,
 "pe": 22.1,
 "roe": 14.2,
 "sales": 600000,
 "profit": 45000
}
{
 "_id": 8,
 "company": "Company-H",
 "sector": "Energy",
 "eps": 6.5,
 "pe": 10.5,
 "roe": 16.4,
 "sales": 550000,
 "profit": 50000
}
```

```
{
 "_id": 9,
 "company": "Company-I",
 "sector": "Consumer Goods",
 "eps": 2.9,
 "pe": 25.3,
 "roe": 7.8,
 "sales": 350000,
 "profit": 20000
}
```

```
db.Stock.insertMany([{ _id: 1, company: "Company-A", sector: "Technology", eps: 5.2, pe: 15.3, roe: 12.8, sales: 300000, profit: 25000 }, { _id: 2, company: "Company-B", sector: "Finance", eps: 7.1, pe: 12.4, roe: 10.9, sales: 500000, profit: 55000 }, { _id: 3, company: "Company-C", sector: "Retail", eps: 3.8, pe: 22.1, roe: 9.5, sales: 200000, profit: 15000 }, { _id: 4, company: "Company-D", sector: "Technology", eps: 5.2, pe: 15.3, roe: 12.8, sales: 300000, profit: 25000 }, { _id: 5, company: "Company-E", sector: "Finance", eps: 7.1, pe: 12.4, roe: 10.9, sales: 450000, profit: 40000 }, { _id: 6, company: "Company-F", sector: "Healthcare", eps: 3.8, pe: 18.9, roe: 9.5, sales: 500000, profit: 35000 }, { _id: 7, company: "Company-G", sector: "Retail", eps: 4.3, pe: 22.1, roe: 14.2, sales: 600000, profit: 45000 }, { _id: 8, company: "Company H", sector: "Energy", eps: 6.5, pe: 10.5, roe: 16.4, sales: 550000, profit: 50000 }, { _id: 9, company: "Company-I", sector: "Consumer Goods", eps: 2.9, pe: 25.3, roe: 7.8, sales: 350000, profit: 20000 }])
```

3. Calculate the total sales of all companies.

```
db.Stock.aggregate([{ $group: { _id: null, totalSales: { $sum: "$sales" } } }])
```

4. Find the average profit for companies in each sector.

```
db.Stock.aggregate([{ $group: { _id: "$sector", avgProfit: { $avg: "$profit" } } }])
```

5. Get the count of companies in each sector.

```
db.Stock.aggregate([{ $group: { _id: "$sector", count: { $sum: 1 } } }])
```

6. Find the company with the highest PE ratio.

```
db.Stock.aggregate([{ $sort: { pe: -1 } }, { $limit: 1 }])
```

7. Filter companies with PE ratio greater than 20.(Use: Aggregate)

```
db.Stock.aggregate([{ $match: { pe: { $gt: 20 } } }])
```

8. Calculate the total profit of companies with sales greater than 250,000.

```
db.Stock.aggregate([{ $match: { sales: { $gt: 250000 } } }, { $group: { _id: null, totalProfit: { $sum: "$profit" } } }])
```

9. Project only the company name and profit fields.(Use: Aggregate)

```
db.Stock.aggregate([{ $project: { company: 1, profit: 1 } }])
```

10. Find companies where EPS is greater than the average EPS.

```
db.Stock.aggregate([{ $group: { _id: null, avgEPS: { $avg: "$eps" } } }, { $lookup: { from: "Stock", let: {
```

avgEPS: "\$avgEPS" }, pipeline: [ { \$match: { \$expr: { \$gt: [ "\$seps", "\$\$avgEPS" ] } } }, as: "companiesWithHigherEPS" } }, { \$unwind: "\$companiesWithHigherEPS" }, { \$replaceRoot: { newRoot: "\$companiesWithHigherEPS" } } ] )

11. Group companies by sector and get the maximum sales in each sector.  
db.Stock.aggregate([ { \$group: { \_id: "\$sector", maxSales: { \$max: "\$sales" } } } ] )
12. Calculate the total sales and total profit of companies in each sector.  
db.Stock.aggregate([ { \$group: { \_id: "\$sector", totalSales: { \$sum: "\$sales" }, totalProfit: { \$sum: "\$profit" } } } ] )
13. Sort companies by profit in descending order.(Use: Aggregate)  
db.Stock.aggregate([ { \$sort: { profit: -1 } } ] )
14. Find the average ROE across all companies.  
db.Stock.aggregate([ { \$group: { \_id: null, avgROE: { \$avg: "\$roe" } } } ] )
15. Group companies by sector and calculate both the minimum and maximum EPS.  
db.Stock.aggregate([ { \$group: { \_id: "\$sector", minEPS: { \$min: "\$seps" }, maxEPS: { \$max: "\$seps" } } } ] )

#### Part – C

1. Count the number of companies with profit greater than 30,000.  
db.Stock.aggregate([ { \$match: { profit: { \$gt: 30000 } } }, { \$count: "companyCount" } ] )
2. Get the total profit by sector and sort by descending total profit.  
db.Stock.aggregate([ { \$group: { \_id: "\$sector", totalProfit: { \$sum: "\$profit" } } }, { \$sort: { totalProfit: -1 } } ] )
3. Find the top 3 companies with the highest sales.  
db.Stock.aggregate([ { \$sort: { sales: -1 } }, { \$limit: 3 } ] )
4. Calculate the average PE ratio of companies grouped by sector.  
db.Stock.aggregate([ { \$group: { \_id: "\$sector", averagePE: { \$avg: "\$pe" } } } ] )
5. Get the sum of sales and profit for each company.  
db.Stock.aggregate([ { \$project: { company: "\$company", totalSalesAndProfit: { \$add: [ "\$sales", "\$profit" ] } } } ] )

|                                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                        | <p>6. Find companies with sales less than 400,000 and sort them by sales.</p> <pre>db.Stock.aggregate([   { \$match: { sales: { \$lt: 400000 } } },   { \$sort: { sales: 1 } } ])</pre> <p>7. Group companies by sector and find the total number of companies in each sector.</p> <pre>db.Stock.aggregate([   { \$group: { _id: "\$sector", totalCompanies: { \$sum: 1 } } } ])</pre> <p>8. Get the average ROE for companies with sales greater than 200,000.</p> <pre>db.Stock.aggregate([   { \$match: { sales: { \$gt: 200000 } } },   { \$group: { _id: null, averageROE: { \$avg: "\$roe" } } } ])</pre> <p>9. Find the maximum profit in each sector.</p> <pre>db.Stock.aggregate([   { \$group: { _id: "\$sector", maxProfit: { \$max: "\$profit" } } } ])</pre> <p>10. Get the total sales and count of companies in each sector.</p> <pre>db.Stock.aggregate([   { \$group: { _id: "\$sector", totalSales: { \$sum: "\$sales" }, companyCount: { \$count: {} } } } ])</pre> <p>11. Project fields where profit is more than 20,000 and only show company and profit.</p> <pre>db.Stock.aggregate([   { \$match: { profit: { \$gt: 20000 } } },   { \$project: { company: 1, profit: 1 } } ])</pre> <p>12. Find companies with the lowest ROE and sort them in ascending order.(Use: Aggregate)</p> <pre>db.Stock.aggregate([   { \$sort: { roe: 1 } } ])</pre> |
| <p><b>Lab-6</b><br/><b>MongoDB</b></p> | <p><b>Perform the following queries on Index, Cursor, Schema Validation, Embedded and Multivalued Documents:</b></p> <p><b>Part – A (Use collection “Stock” created in Lab-21)</b></p> <p>1. Create an index on the company field in the stocks collection.</p> <pre>db.Stock.createIndex({ company: 1 })</pre> <p>2. Create a compound index on the sector and sales fields in the stocks collection.</p> <pre>db.Stock.createIndex({ sector: 1, sales: -1 })</pre> <p>3. List all the indexes created on the stocks collection.</p> <pre>db.Stock.getIndexes()</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |



4. Drop an existing index on the company field from the stocks collection.  
`db.Stock.dropIndex("company_1")`
5. Use a cursor to retrieve and iterate over documents in the stocks collection, displaying each document.  
`const cursor = db.Stock.find();  
cursor.forEach(doc => printjson(doc))`
6. Limit the number of documents returned by a cursor to the first 3 documents in the stocks collection.  
`const cursor = db.Stock.find().limit(3);  
cursor.forEach(doc => printjson(doc));`
7. Sort the documents returned by a cursor in descending order based on the sales field.  
`const cursor = db.Stock.find().sort({ sales: -1 });  
cursor.forEach(doc => printjson(doc));`
8. Skip the first 2 documents in the result set and return the next documents using the cursor.  
`const cursor = db.Stock.find().skip(2);  
cursor.forEach(doc => printjson(doc));`
9. Convert the cursor to an array and return all documents from the stocks collection.  
`const allDocsArray = db.Stock.find().toArray();  
printjson(allDocsArray);`
10. Create a collection named "Companies" with schema validation to ensure that each document must contains a company field (string) and a sector field (string).  
`db.createCollection("Companies", {  
 validator: {  
 $jsonSchema: {  
 bsonType: "object",  
 required: ["company", "sector"],  
 properties: {  
 company: {  
 bsonType: "string",  
 description: "must be a string and is required"  
 },  
 sector: {  
 bsonType: "string",  
 description: "must be a string and is required"  
 }  
 }  
 }  
 }  
});`

**Part – B**

1. Create a collection named "Scripts" with validation for fields like eps, pe, and roe to ensure that they are numbers and required/compulsory fields.

```
db.createCollection("Scripts", {
 validator: {
 $jsonSchema: {
 bsonType: "object",
 required: ["eps", "pe", "roe"],
 properties: {
 eps: {
 bsonType: "number",
 description: "must be a number and is required"
 },
 pe: {
 bsonType: "number",
 description: "must be a number and is required"
 },
 roe: {
 bsonType: "number",
 description: "must be a number and is required"
 }
 }
 }
 }
});
```

2. Create a collection named "Products" where each product has an embedded document for manufacturer details and a multivalued field for categories that stores an array of category names the product belongs to.

- manufacturer details: The manufacturer will be an embedded document with fields like name, country, and establishedYear.
- categories: The categories will be an array field that holds multiple values. (i.e. Electronics, Mobile, Smart Devices).

```
db.createCollection("Products", {
 validator: {
 $jsonSchema: {
 bsonType: "object",
 required: ["manufacturer", "categories"],
 properties: {
 manufacturer: {
 bsonType: "object",
 required: ["name", "country", "establishedYear"],
 properties: {
 name: {
 bsonType: "string",
 description: "must be a string and is required"
 },
 country: {
 bsonType: "string",
 description: "must be a string and is required"
 }
 }
 },
 categories: {
 bsonType: "array",
 description: "must be an array and is required"
 }
 }
 }
 }
});
```

```

 establishedYear: {
 bsonType: "int",
 description: "must be an integer and is required"
 }
 },
 categories: {
 bsonType: "array",
 items: {
 bsonType: "string",
 description: "must be a string"
 },
 description: "must be an array of strings and is required"
 }
}
}
}
});

```

#### Part – C

1. Create a collection named "financial\_Reports" that requires revenue (a positive number) but allows optional fields like expenses and netIncome (if provided, they should also be numbers).

```

db.createCollection("financial_Reports", {
 validator: {
 $jsonSchema: {
 bsonType: "object",
 required: ["revenue"],
 properties: {
 revenue: {
 bsonType: "double",
 description: "must be a positive number and is required"
 },
 expenses: {
 bsonType: "double",
 description: "must be a number if provided"
 },
 netIncome: {
 bsonType: "double",
 description: "must be a number if provided"
 }
 }
 }
 }
});

```

2. Create a collection named "Student" where each student has name and address are embedded document and mobilenumber and emailaddress are multivalued field that stores an array of values.

```

db.createCollection("Student", {

```

```
validator: {
 $jsonSchema: {
 bsonType: "object",
 required: ["name", "address", "mobileNumber", "emailAddress"],
 properties: {
 name: {
 bsonType: "string",
 description: "must be a string and is required"
 },
 address: {
 bsonType: "object",
 properties: {
 street: { bsonType: "string" },
 city: { bsonType: "string" },
 state: { bsonType: "string" },
 zip: { bsonType: "string" }
 }
 },
 mobileNumber: {
 bsonType: "array",
 items: { bsonType: "string" },
 description: "must be an array of strings and is required"
 },
 emailAddress: {
 bsonType: "array",
 items: { bsonType: "string" },
 description: "must be an array of strings and is required"
 }
 }
 }
};
```